

Task 1: German Tank Problem

COURSE NAME

CSDS 413 Introduction to Data Analysis

Authors:

Jacob Anderson

October 13, 2025

Contents

Context	2
1 Part A	3
2 Part B	4
3 Part C	5

Context

Recall the German Tank (or the “Rocket Science”) problem from Exercise #7 in the class. You know that the Nazi Army is assigning sequential integer IDs to their tanks. In other words, if they have M tanks, there is a tank with ID i for all $1 \leq i \leq M$. The allied forces capture n tanks and the IDs of these tanks are $1 \leq x_1 \leq x_2 \leq \dots \leq x_n$. Our goal is to estimate M , i.e., the number of tanks that the Nazis have, based on the assumption that the tanks were captured uniformly at random from these M tanks.

1 Part A

Let $\hat{M}_{MLE}$ be the maximum likelihood estimator for M . Compute $\hat{M}_{MLE}$.

Under the assumption in the context, "the tanks were captured uniformly at random from these M tanks", we are looking at some unique list of n tank IDs from $\binom{M}{n}$ possible observations, making the probability of an individual combination of n tank IDs captured $\frac{1}{\binom{M}{n}}$.

However, the likelihood of this observation is non-zero under the condition that M is greater than or equal to x_n , or else it would be impossible to observe x_n under this capturing assumption:

$$L(M) = \begin{cases} \frac{1}{\binom{M}{n}} & \text{if } M \geq x_n \\ 0 & \text{if } M < x_n \end{cases}$$

Regarding the maximum, likelihood is zero for M less than x_n , and when M is greater than x_n , $\binom{M}{n}$ becomes larger and makes our specific observation of IDs less likely given a random, uniform sampling, meaning the likelihood must decrease for increasing M , so the MLE estimator must take on the most constraining value of M that does not make the likelihood of the observation impossible, which is the maximum observed ID: $\hat{M}_{MLE} = x_n = \max(x_1, x_2, \dots, x_n)$.

Below is the implementation of the estimator for M :

```
def tank_capture(M: int, n: int) -> list:
    """
    Capture n tank IDs uniformly at random from M total tanks.

    :param M: Total number of tanks
    :type M: int
    :param n: Number of tanks captured
    :type n: int
    :returns: List of captured tank IDs
    :rtype: list
    """
    return sorted(np.random.choice(range(1, M + 1), size=n, replace=False).tolist())

def mle_M(tank_ids: list) -> int:
    """
    Computes the MLE for the total number of tanks given
    an observation of tank IDs.

    :param tank_ids: List of observed tank IDs
    :type tank_ids: list
    :returns: Maximum likelihood estimate of total tanks
    :rtype: int
    """
    return max(tank_ids)
```

Figure 1: Problem setup and MLE computation.

Here is an arbitrary example of $(M = 100, n = 36)$, just to exemplify the computation:

```
(assignment2-py3.12) jande@BT-7274:~/csds413_assignments/
assignment2/task1_german_tank/scripts$ python
Python 3.12.0 (main, Jun 9 2025, 09:52:12) [GCC 13.3.0] on linux
Type "help", "copyright", "credits" or "license" for more information.
>>> from estimators import mle_M
>>> from sim import tank_capture
>>> captured_ids = tank_capture(100, 36)
>>> print(f"Captured tank IDs: {captured_ids}")
Captured tank IDs: [1, 4, 8, 9, 10, 12, 16, 18, 19, 20, 27, 28, 31,
43, 44, 45, 50, 52, 56, 58, 61, 62, 65, 68, 70, 71, 72, 78, 79, 81,
83, 84, 86, 94, 95, 97]
>>> print(f"MLE of total tanks: {mle_M(captured_ids)}")
MLE of total tanks: 97
```

2 Part B

Let $\hat{M}_{MEAN} = 2(\sum_{i=1}^n x_i/n) - 1$ and $\hat{M}_{MVU} = x_n(\frac{n+1}{n}) - 1$ be two additional candidate estimators for M . Theoretically characterize the expected value of each of the three estimators ($\hat{M}_{MLE}$, $\hat{M}_{MEAN}$, $\hat{M}_{MVU}$) for a given M and N . Comment on the bias of each estimator.

These expected values for each estimator were given in the recent Canvas announcement:

$$\begin{aligned} E[\hat{M}_{MLE}] &= \frac{n}{n+1}(M+1) \\ E[\hat{M}_{MEAN}] &= M \\ E[\hat{M}_{MVU}] &= M \end{aligned}$$

Generally speaking, the bias of an estimator is the difference between the expected value of the estimator and the true parameter value. That makes $E[\hat{M}_{MEAN}] = M$ and $E[\hat{M}_{MVU}] = M$ very straightforward cases: $M - M = 0$; these two have no bias, they are unbiased estimators. The MEAN estimator correctly estimates M on average by using the sample mean of observed tank IDs because under uniform sampling, the average captured ID value should approximate the true mean of the tank population. The MVU estimator achieves M on average by applying $(n+1)/n$ to the maximum observed value as a correction factor to the undervaluation by the MLE.

Regarding the bias of $\hat{M}_{MLE}$ as $E[\hat{M}_{MLE}] - M = \frac{n}{n+1}(M+1) - M$, this can be simplified to $-\frac{M-n}{n+1}$:

$$\begin{aligned} E[\hat{M}_{MLE}] - M &= \frac{n}{n+1}(M+1) - M \\ &= \frac{n(M+1) - M(n+1)}{n+1} \\ &= \frac{nM + n - Mn - M}{n+1} \\ &= \frac{n - M}{n+1} \\ &= -\frac{M - n}{n+1} \end{aligned}$$

The bias is negative, so on average the MLE will underestimate the total number of tanks. That reflects well how the MLE computation works, because the only cases where MLE is M are the cases where the random, uniform sample of n tanks includes the highest-numbered tank; otherwise, the MLE will be off by the difference between M and the maximum value observed in the sample.

3 Part C

Using different values of M and n , simulate different instances of the German Tank Problem, visualize the mean and variance of these three estimators ($\hat{M}_{MLE}$, $\hat{M}_{MEAN}$, $\hat{M}_{MVU}$) as a function of M and n (i.e., fix M to, say 100, 1000, 10K etc., and for each M , plot the mean and variance of the estimators as a function of n - you can compute the mean and variance by repeating the simulation multiple times for each combination of M and n). Discuss the differences between the estimators in terms of their bias and variance.

I performed simulations of the German Tank Problem for each of these M values: 100, 1000, 10000. For each value, I varied n from 5% to 95% of each population, in 5% increments. For each (M, n) combination, the three estimators were computed across 1000 simulations and mean and variance were collected across the simulations.

Figure 2 illustrates all of the code related to computing each of the three estimators:

```
def mle_M(tank_ids: list) -> int:
    """
    Computes the MLE for the total number of tanks.

    :param tank_ids: List of observed tank IDs
    :type tank_ids: list
    :returns: Maximum likelihood estimate of total tanks
    :rtype: int
    """
    return max(tank_ids)

def mean_M(tank_ids: list) -> float:
    """
    Computes the MEAN estimator for the total number of tanks.

    :param tank_ids: List of observed tank IDs
    :type tank_ids: list
    :returns: MEAN estimate of total tanks
    :rtype: float
    """
    return 2 * np.mean(tank_ids) - 1

def mvu_M(tank_ids: list) -> float:
    """
    Computes the MVU estimator for the total number of tanks.

    :param tank_ids: List of observed tank IDs
    :type tank_ids: list
    :returns: MVU estimate of total tanks
    :rtype: float
    """
    n = len(tank_ids)
    return max(tank_ids) * ((n + 1) / n) - 1
```

Figure 2: Estimator functions for $\hat{M}_{MLE}$, $\hat{M}_{MEAN}$, $\hat{M}_{MVU}$.

The simulations were handled by a module containing, alongside `tank_capture()` from Part A, a function for simulating repeated trials of a specific (M, n) configuration of the German Tank Problem and a function for coordinating all of the different (M, n) configurations, shown below:

```
def simulate_estimators(M: int, n: int, num_trials: int) -> dict:
    """
    Simulates the German Tank Problem multiple times and computes
    all three estimators for each.

    :param M: Total number of tanks
```

```

:type M: int
:param n: Number of tanks to capture per sim
:type n: int
:param num_trials: Number of simulations
:type num_trials: int
:returns: Dictionary with lists of estimates for each estimator
:rtype: dict
"""
mle_estimates = []
mean_estimates = []
mvu_estimates = []

for _ in range(num_trials):
    captured = tank_capture(M, n)
    mle_estimates.append(mle_M(captured))
    mean_estimates.append(mean_M(captured))
    mvu_estimates.append(mvu_M(captured))

return {
    'mle': mle_estimates,
    'mean': mean_estimates,
    'mvu': mvu_estimates
}

def run_simulations(M_values: list, num_trials: int) -> pd.DataFrame:
    """
    Runs simulations for all (M, n) combinations,
    computing mean and variance of each estimator.

    :param M_values: List of M values to test
    :type M_values: list
    :param num_trials: Number of trials per (M, n) combination
    :type num_trials: int
    :returns: DataFrame with simulation results
    :rtype: pd.DataFrame
    """
    n_percentages = [p * 0.01 for p in range(5, 100, 5)]
    results = []

    for M in M_values:
        n_values = [max(1, int(M * p)) for p in n_percentages]

        for n in n_values:
            estimates = simulate_estimators(M, n, num_trials)

            results.append({
                'M': M,
                'n': n,
                'mle_mean': np.mean(estimates['mle']),
                'mle_var': np.var(estimates['mle']),
                'mean_mean': np.mean(estimates['mean']),
                'mean_var': np.var(estimates['mean']),
                'mvu_mean': np.mean(estimates['mvu']),
                'mvu_var': np.var(estimates['mvu'])
            })

    df = pd.DataFrame(results)
    return df

```

Figure 3: Simulation functions for comparing the three estimators.

Below is the visualization code used to generate the comparison plots for estimator mean and variance:

```

def plot_estimator_means(results_df: pd.DataFrame, output_dir: str):
    """
    Plots the mean of each estimator as a function of n, for each M value.

    :param results_df: DataFrame of estimator means and variances
    :type results_df: pd.DataFrame
    :param output_dir: Where to save the figures
    :type output_dir: str
    """
    sns.set_style("whitegrid")
    sns.set_palette("muted")

    M_values = results_df['M'].unique()

    for M in M_values:
        df = results_df[results_df['M'] == M]

        plt.figure(figsize=(12, 7))

        ax = plt.gca()
        ax.plot(df['n'], df['mle_mean'],
                marker='o', markersize=4, label='MLE', linewidth=2, alpha=0.8)
        ax.plot(df['n'], df['mean_mean'],
                marker='o', markersize=4, label='MEAN', linewidth=2, alpha=0.8)
        ax.plot(df['n'], df['mvu_mean'],
                marker='o', markersize=4, label='MVU', linewidth=2, alpha=0.8)
        ax.axhline(y=M, color='black', linestyle='--', linewidth=1.5,
                    label=f'True M={M}')

        ax.set_xlabel('Sample Size n', fontsize=14, labelpad=15)
        ax.set_ylabel('Mean Estimate', fontsize=14, labelpad=15)
        ax.set_title(f'Mean of Estimators vs Sample Size (M={M})', fontsize=16, pad=20)

        legend = ax.legend(fontsize=12, loc='best')
        for text in legend.get_texts():
            text.set_fontsize(11)

        plt.tight_layout()
        os.makedirs(output_dir, exist_ok=True)
        plt.savefig(f'{output_dir}/mean_comparison_M{M}.png', bbox_inches='tight', dpi=300)

def plot_estimator_variances(results_df: pd.DataFrame, output_dir: str = '../figures'):
    """
    Plots the variance of each estimator as a function of n, for each M value.

    :param results_df: DataFrame from estimator means and variances
    :type results_df: pd.DataFrame
    :param output_dir: Where to save the figures
    :type output_dir: str
    """
    sns.set_style("whitegrid")
    sns.set_palette("muted")

    M_values = results_df['M'].unique()

    for M in M_values:
        df = results_df[results_df['M'] == M]

        plt.figure(figsize=(12, 7))

        ax = plt.gca()

```

```

ax.plot(df['n'], df['mle_var'],
        marker='d', markersize=8, label='MLE', linewidth=2, alpha=0.8)
ax.plot(df['n'], df['mean_var'],
        marker='o', markersize=4, label='MEAN', linewidth=2, alpha=0.8)
ax.plot(df['n'], df['mvu_var'],
        marker='o', markersize=4, label='MVU', linewidth=2, alpha=0.8)

ax.set_xlabel('Sample Size n', fontsize=14, labelpad=15)
ax.set_ylabel('Variance', fontsize=14, labelpad=15)
ax.set_title(f'Variance of Estimators vs Sample Size (M={M})', fontsize=16, pad=20)

legend = ax.legend(fontsize=12, loc='best')
for text in legend.get_texts():
    text.set_fontsize(11)

plt.tight_layout()
os.makedirs(output_dir, exist_ok=True)
plt.savefig(f'{output_dir}/variance_comparison_M{M}.png', bbox_inches='tight', dpi=300)

```

Figure 4: Visualization functions for estimator comparison.

You'll notice that in the variance figures, I chose to make the markers associated with the MLE plots larger and in the shape of diamonds because at high population scales, the plots of MLE (blue) and MVU (green) variance are so similar to each other, relative to MEAN, that they almost perfectly overlap each other visually. This was intended to make sure the reader can still recognize the individual plots in the figures.

Figures 5-7 show the mean of each estimator as a function of sample size n for $M = 100, 1000, 10000$.

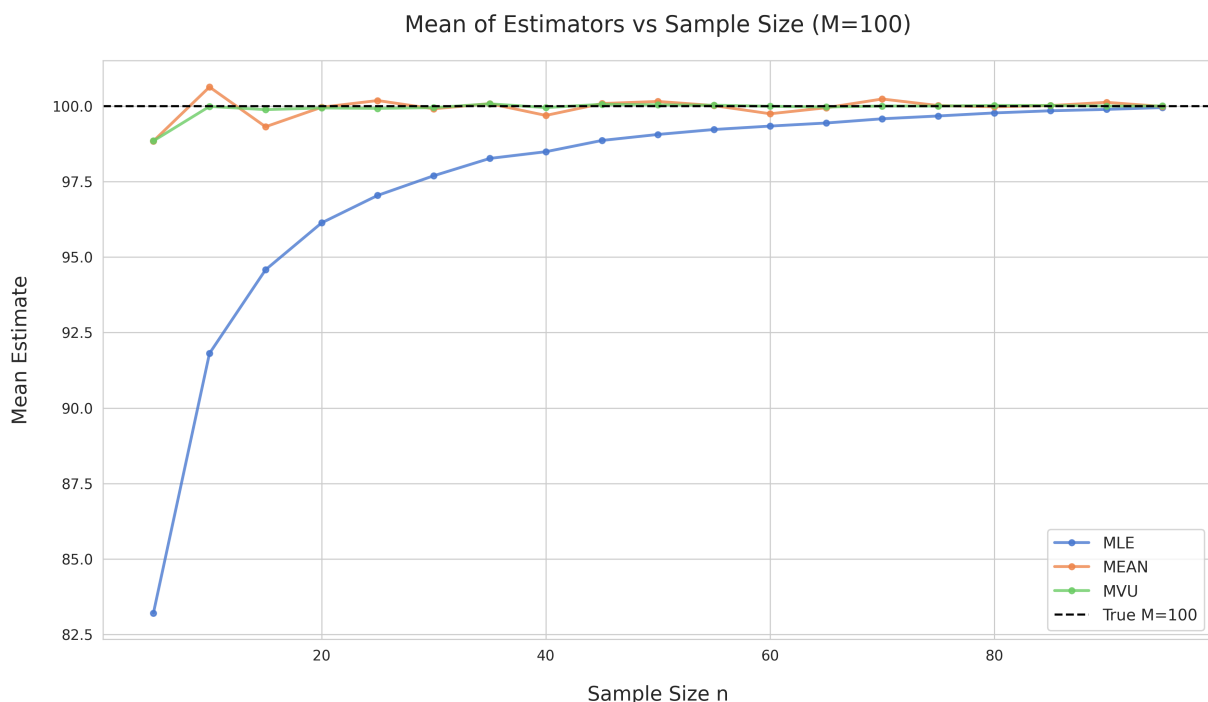


Figure 5: Mean of estimators vs sample size for $M = 100$.

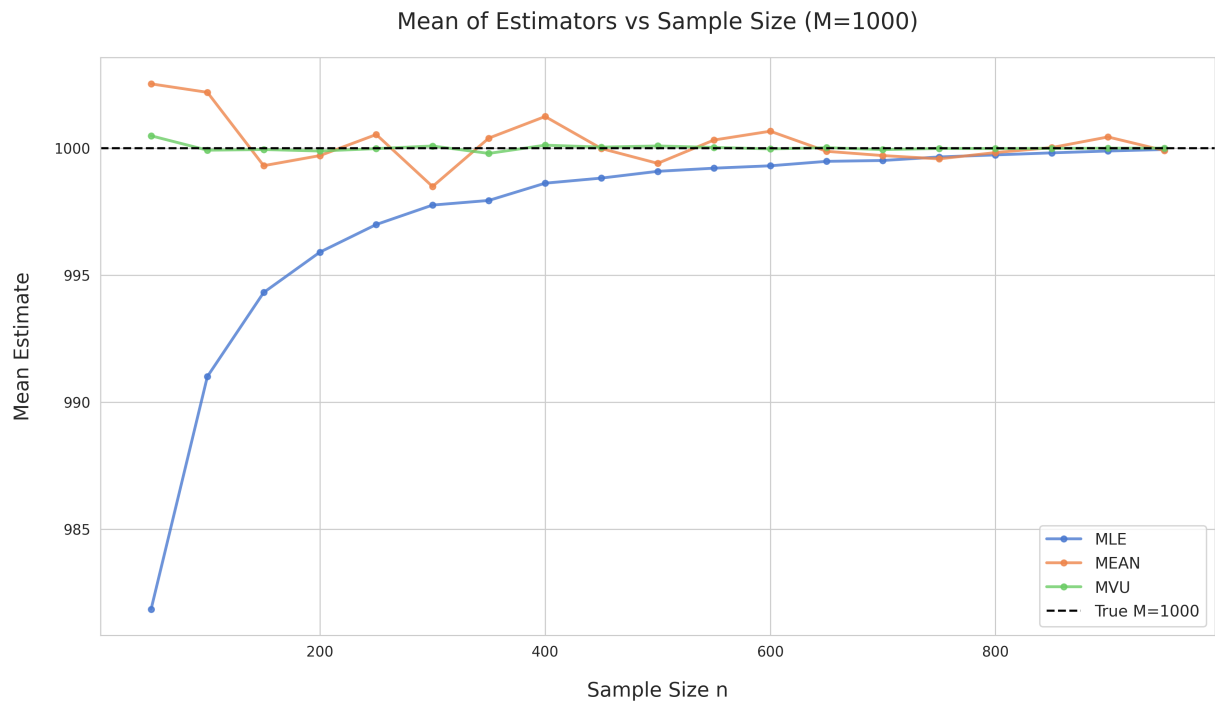


Figure 6: Mean of estimators vs sample size for $M = 1000$.

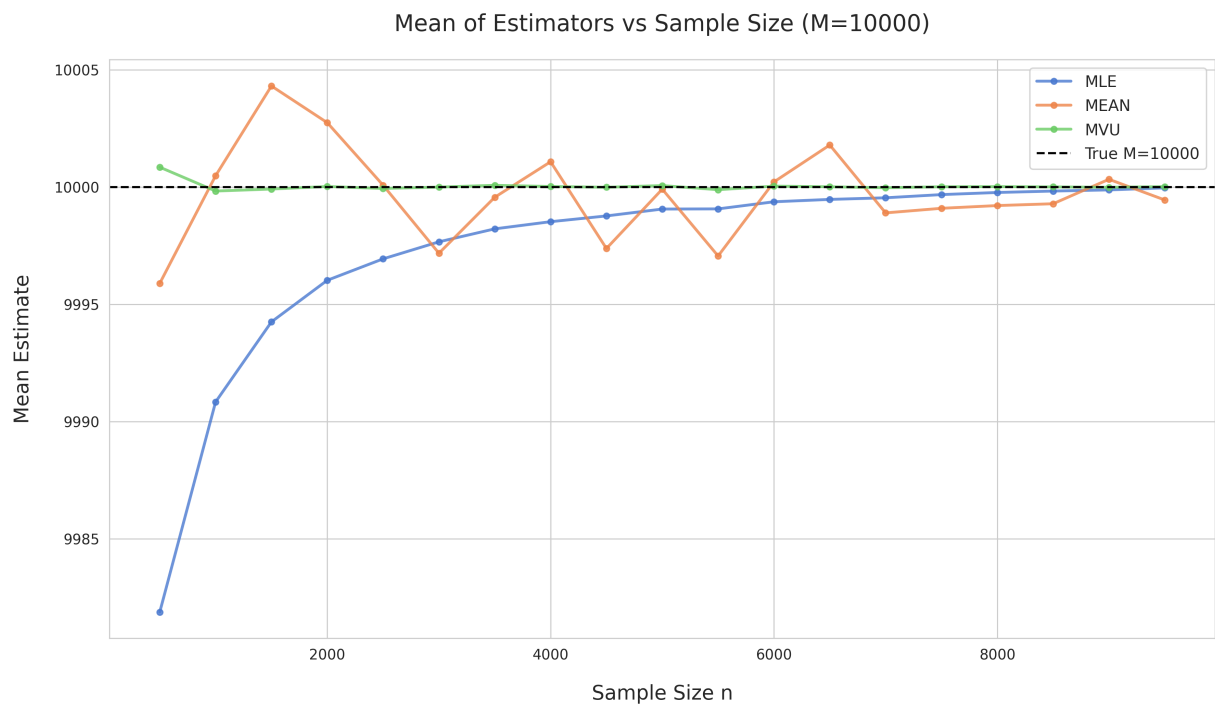


Figure 7: Mean of estimators vs sample size for $M = 10000$.

Figures 8-10 show the variance of each estimator as a function of sample size n for the same values of M .

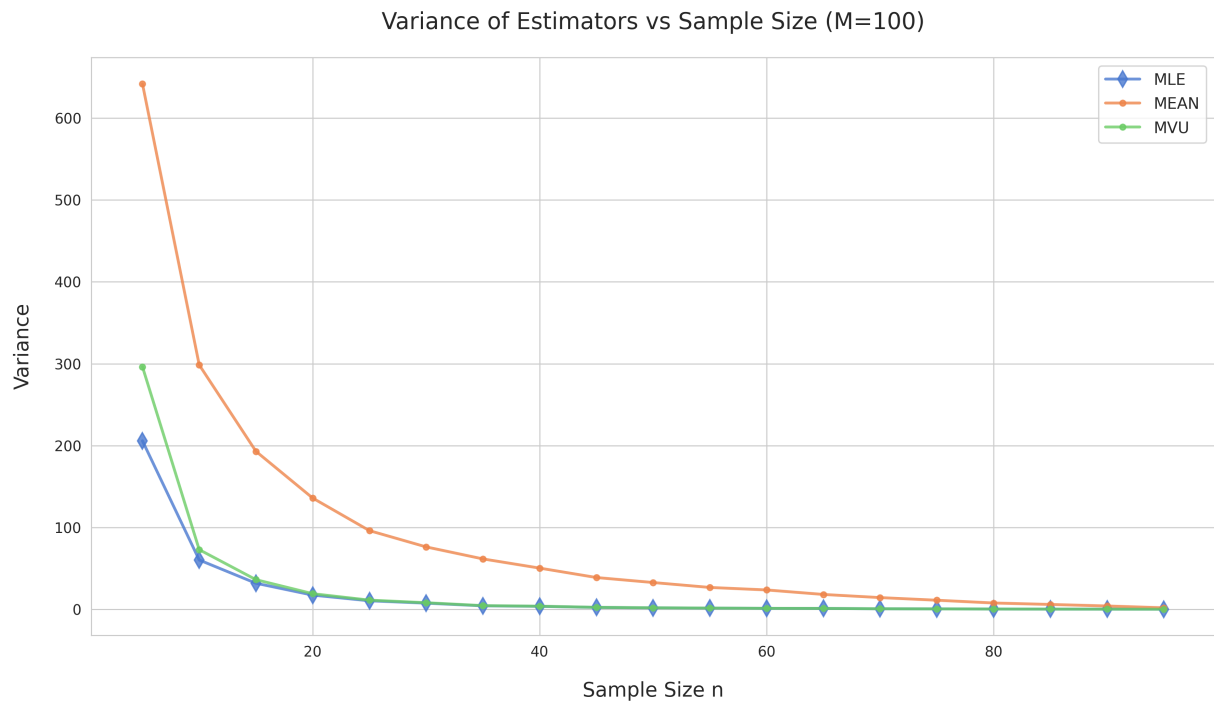


Figure 8: Variance of estimators vs sample size for $M = 100$.

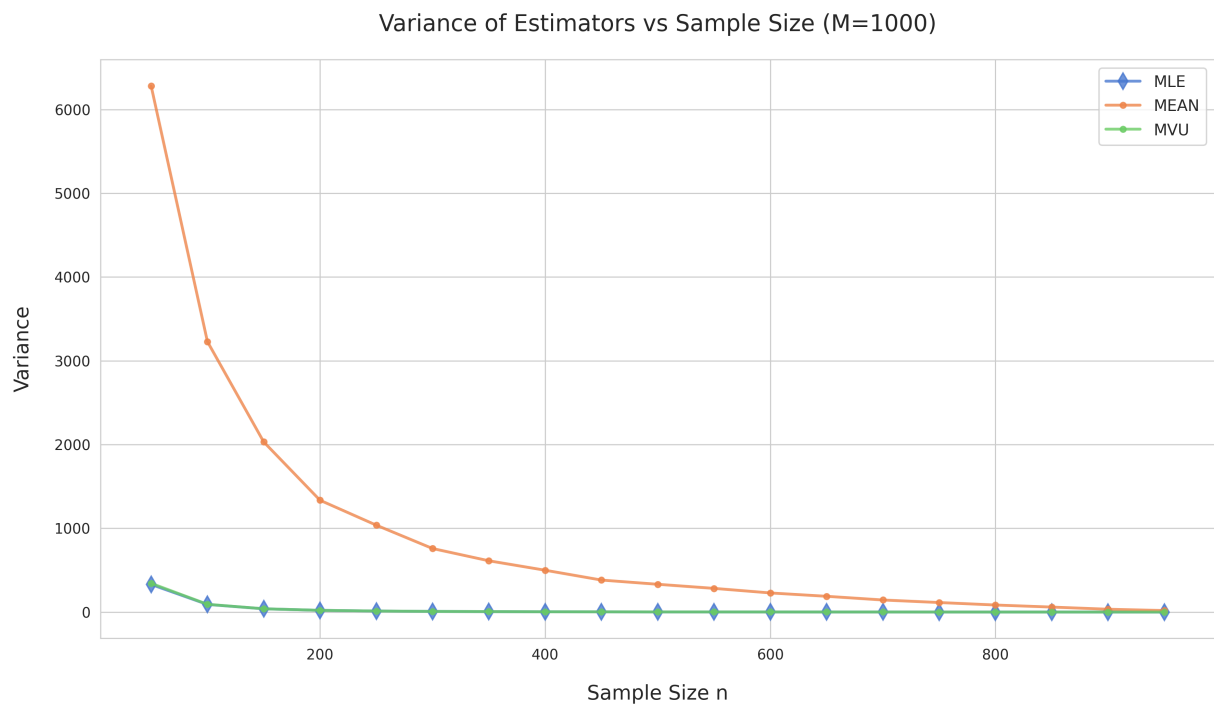


Figure 9: Variance of estimators vs sample size for $M = 1000$.

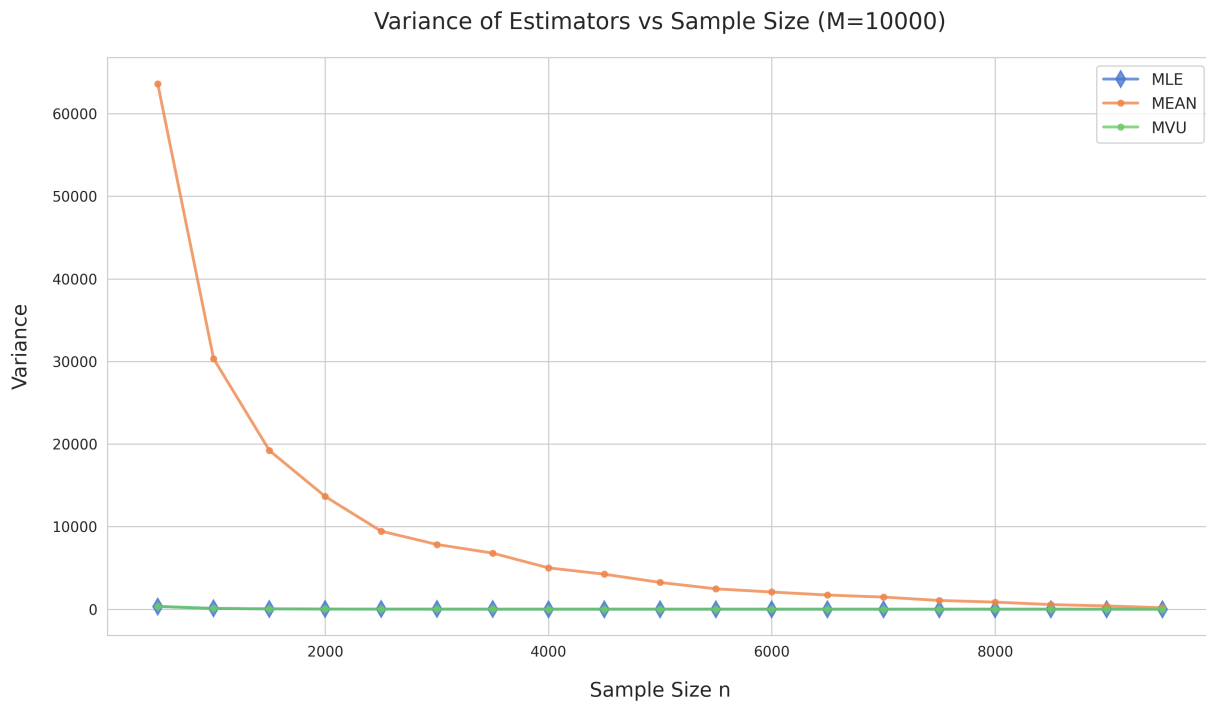


Figure 10: Variance of estimators vs sample size for $M = 10000$.

The plots of estimator mean in each of the three figures clearly illustrate the unique bias of the MLE for this German Tank Problem, where it consistently underestimates the true value of M . This estimate then becomes more and more accurate to the true population as n increases, approaching the dashed line representing true M in the figure. We can think of that behavior intuitively as the uniform, random sampling having more opportunities to observe a value closer to the actual maximum ID present for the population.

The correction factor applied for MVU yields a very satisfying outcome, as the plot quickly converges to the true parameter value, riding the dashed line at true M across the majority of the sample sizes. The MVU estimate is calibrated exactly as it needs to be to adjust for the structural underestimation present in the MLE, because the correction factor $\frac{n+1}{n}$ is literally the inverse of the factor underlying the undervaluation of the MLE, expressed in its expectation and is what we observed in the calculation of MLE bias in Part B: $E[\hat{M}_{MLE}] - M = \frac{n}{n+1}(M+1) - M$. In the case of MVU, the expected value simplifies to $E[\hat{M}_{MVU}] - M = \frac{n+1}{n} \cdot \frac{n}{n+1}(M+1) - M = (M+1) - M$. I'm assuming that similar to what we discussed in lecture last week, the convention of the $+1$ factor in this final expression is sometimes omitted sometimes not, and for large M , is very insignificant, though regardless the expected value of MVU was provided by the professor as M . In any case, the MVU is corrected by exactly what is necessary to fully address the negative bias present in the MLE.

MEAN is also unbiased like MVU, however it's visually clear from the estimator mean figures that the underlying mechanism that makes MEAN unbiased is not nearly as well-calibrated for smaller n . MEAN is unbiased in the sense that the sample mean of the observations will approximate the population mean and will become more representative of the population mean for greater numbers of samples.

That being said, this approximation is more sensitive to the uniform, random sampling for small n , as opposed to the careful correction of MVU with respect to sample size, $\frac{n+1}{n}$. For this reason, we can see in the variance figures for any of the M values that as n decreases, the MEAN estimate skyrockets in variance compared to the MVU estimate because the estimate's stability is dependent on the random chance that the individual sets of observations over the repeated simulations happen to yield similar results.

Regarding the variance of the MLE, we can now verify visually that variance is quite low in each of the cases of M and across the different sample sizes n . While structurally biased, the design of MLE yields consistently close estimates given some set number of chances to sample a value close to M . In other words, its inconsistency in estimating M depends only on how many chances the sampling is given to observe values close to M , and so as n increases, the estimator converges quite tightly to M .

Task 2: The Distribution of the Sample Mean

COURSE NAME

CSDS 413 Introduction to Data Analysis

Authors:

Jacob Anderson

October 13, 2025

Contents

Context	2
1 Part A: Data Generation	3
2 Part B: Sample Means and Visualization	7
3 Part C: Normal Approximation via CLT	12
4 Part D: Distribution Comparison with KS-Test	17

Context

In this task, you will we gain insight about the Central Limit Theorem. Consider three random variables:

- (a) A **Uniform distribution** on $[0.25, 1.25]$
- (b) A **Power-Law distribution** (with parameter $\alpha = 3$)

1 Part A: Data Generation

- For each distribution, generate **replicates of data** with sizes 10, 100, and 1000.
- Each replicate should consist of 100 samples.
- Example: For “10 replicates,” generate 10 separate datasets, each with 100 samples.
- For one replicate from each distribution, plot the distribution of the sample to ensure that it follows a uniform/power-law distribution.

Implementation notes:

- In **Python**, use `numpy.random.uniform` and `numpy.random.power`.
- In **R**, use `runif()`, `rplcon()` (for Power, with $\text{shape1} = \alpha$, $\text{shape2} = 2.5$).

For each distribution, the replicates were generated according to the recommended NumPy functions, and in the case of the second distribution, I took the guidance of the Canvas announcement and used the Pareto distribution from `numpy.random.pareto` with α parameter set to 2.3. My implementation of the generation functions just wrap the corresponding NumPy calls that let me set the shape according to the replicate sizes. So for the replicate with size 10, I have a CSV with 10 columns, one for each set of 100 samples. These are the relevant utility functions:

```
def generate_uniform_replicates(low: float, high: float, sample_size: int,
                               num_replicates: int) -> np.ndarray:
    """
    Generates the replicates of uniformly distributed data.

    :param low: Lower bound of uniform distribution
    :type low: float
    :param high: Upper bound of uniform distribution
    :type high: float
    :param sample_size: Number of samples per replicate
    :type sample_size: int
    :param num_replicates: Number of replicates to generate
    :type num_replicates: int
    :returns: Array containing the replicates.
    :rtype: np.ndarray
    """
    replicates = np.random.uniform(low=low, high=high,
                                   size=(num_replicates, sample_size))
    return replicates

def generate_powerlaw_replicates(sample_size: int,
                                 num_replicates: int) -> np.ndarray:
    """
    Generates the replicates of power-law distributed data using Pareto distribution.

    :param alpha: Shape parameter for the power-law distribution
    :type alpha: float
    :param sample_size: Number of samples per replicate
    :type sample_size: int
    :param num_replicates: Number of replicates to generate
    :type num_replicates: int
    :returns: Array containing the replicates.
    :rtype: np.ndarray
    """
    replicates = np.random.pareto(a=2.3, size=(num_replicates, sample_size))
    return replicates
```

Figure 1: Data generation functions for Uniform and Power-Law distributions.

To verify that the generated replicates follow their expected distributions, I plotted the distribution of a single replicate for both the distributions. The visualization function and corresponding plots are shown below:

```
def plot_single_replicate(data: np.ndarray, dist_name: str,
                        output_filename: str):
    """
    Plots the distribution of a single replicate.

    :param data: Single replicate samples
    :type data: np.ndarray
    :param dist_name: Name of the distribution
    :type dist_name: str
    :param output_filename: Filename for the figure
    :type output_filename: str
    """
    sns.set_style("whitegrid")
    sns.set_palette("muted")
    plt.figure(figsize=(10, 6))

    ax = sns.histplot(data, bins=50, stat='density',
                      edgecolor='white', alpha=0.7)

    ax.set_xlabel('Value', fontsize=14, labelpad=15)
    ax.set_ylabel('Density', fontsize=14, labelpad=15)
    ax.set_title(f'Distribution of {dist_name} Replicate',
                 fontsize=16, pad=20)

    plt.tight_layout()
    plt.savefig(f"../figures/{output_filename}", bbox_inches='tight', dpi=300)
```

Figure 2: Visualization function for verifying the distribution shape.

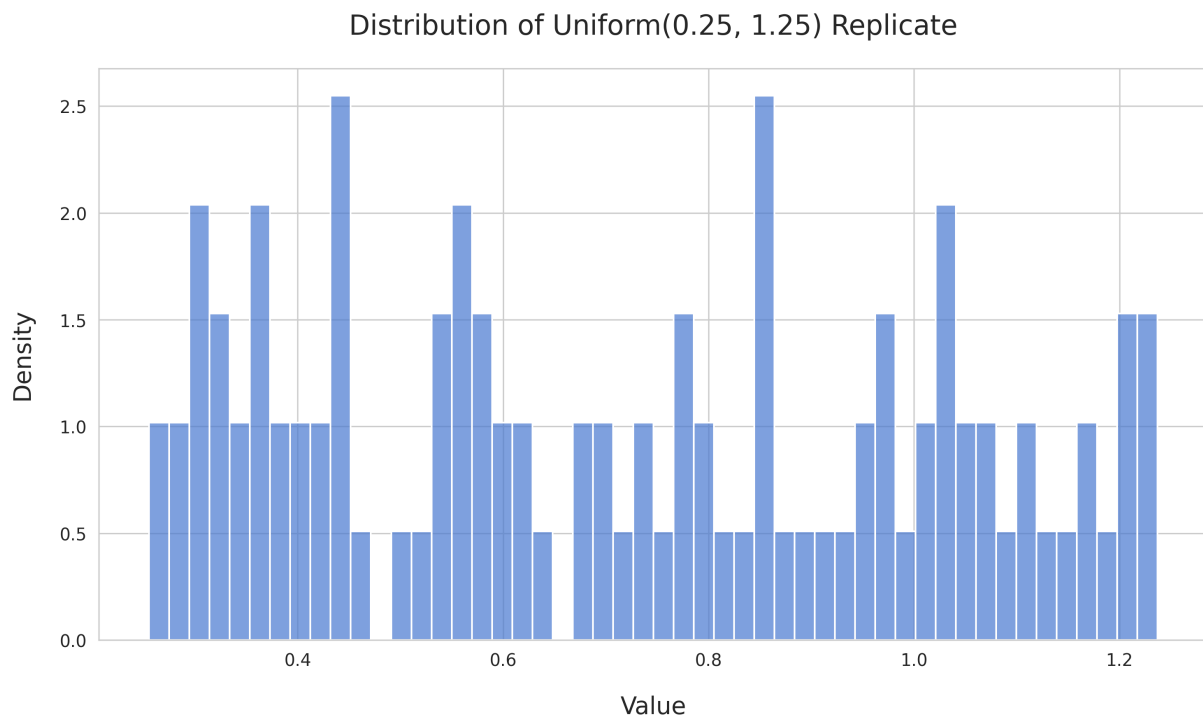


Figure 3: Distribution of a replicate from the Uniform(0.25, 1.25) distribution.

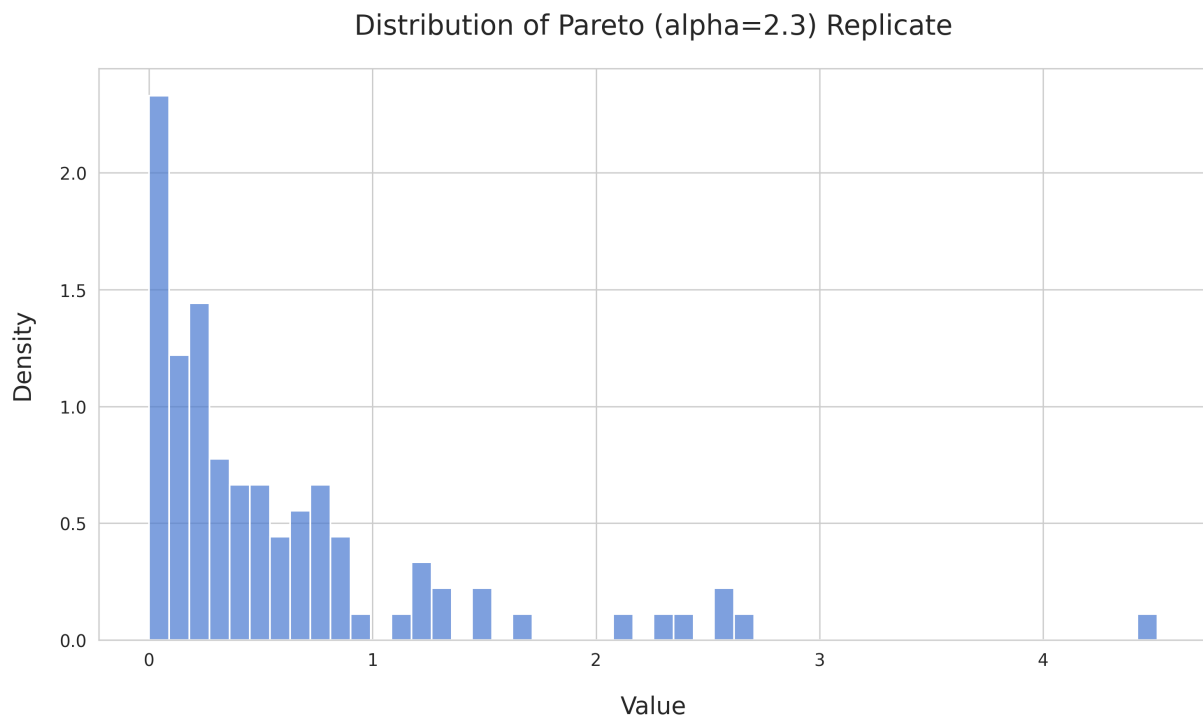


Figure 4: Distribution of a replicate from the Pareto($\alpha = 2.3$) distribution.

The Uniform replicate plot in Figure 3 shows the generally flat distribution of mass that we would expect across the interval, at least as flat as we could reasonably expect from only 100 samples.

The Pareto replicate plot in Figure 4 exhibits the characteristic right-skewed shape of a heavy-tailed power-law distribution, like the ones we have discussed in lecture. Most of the distribution mass is concentrated near zero, and a long tail extends out to larger values, showing that generally the probability density decreases with larger values, but extreme values are still present and predictable in the sample.

Regarding `np.random.power` and `np.random.pareto`, the Canvas announcement made the clarification that we are meant to explore the discrepancy in the distribution shapes these two approaches produce. According to the NumPy documentation, `np.random.power` draws samples from a power function distribution with positive exponent $a - 1$. The probability density function is:

$$P(x; a) = a \cdot x^{a-1}, \quad 0 \leq x \leq 1, \quad a > 0$$

Also I feel I should mention that the documentation literally states, "The power function distribution is just the inverse of the Pareto distribution". So given that guidance and looking at our case in the assignment, alpha is meant to be set to 3 if using `numpy.random.power`, making the PDF:

$$P(x; a) = 3x^2, \quad 0 \leq x \leq 1$$

There are definitely a lot of characteristics that distinguish this equation from the power-laws we have looked at in the lecture as well as the one defined by `numpy.random.pareto` (bounded interval, increasing density instead of decreasing, etc.), but let's look first at the Pareto distribution PDF. `np.random.pareto(a)` samples from a Pareto Type II distribution with shape parameter a and implicit scale $m = 1$, which has the probability density function:

$$p(x) = \frac{a}{(1+x)^{a+1}}, \quad x \geq 0$$

Applying our setting of $a = 2.3$ gives the PDF $p(x) = \frac{2.3}{(1+x)^{3.3}}$. So the `np.random.power` approach is described by $3x^2$ on $[0, 1]$, where the exponent is positive and the function value increases as x approaches 1, concentrating almost all probability mass near the upper bound with no capacity for extreme outliers beyond this range. In contrast, `np.random.pareto` is described by $\frac{2.3}{(1+x)^{3.3}}$ on $[0, \infty)$, where the function value decreases as x increases (since $(1+x)^{3.3}$ in the denominator grows), placing most mass near zero but maintaining enough probability in the tail to produce occasional extreme values.

This distribution produced by `numpy.random.power` seems distinctly different from what we have discussed in lecture. After having applied the settings given in the assignment instructions, the inverse relationship and subsequent effect on the distribution shape was readily obvious, but that still did not explain why `numpy.random.power` is set up in this manner; it seems inconsistent with what I have come to understand Power-Law to be from class. My understanding from the documentation and community support online has led me to believe that `numpy.random.power` generates from a power function distribution, with an exponential relationship to its alpha parameter, which is inconsistent with the general power-law relationship that we established in lecture; that is, $p(x) \propto x^{-a}$. The Pareto Type II distribution from `np.random.pareto`, on the other hand, exhibits the characteristic heavy-tail behavior where the denominator $(1 + x)^{3.3}$ creates the decreasing density function consistent with power-law relationship. They are related but distinctly different in their construction and produce starkly different distribution shapes as a result.

2 Part B: Sample Means and Visualization

- Compute the sample means for each set of replicates. You should have 9 of them, 3 for each distribution with varying sample sizes.
- For each distribution and replicate size, we want to view and compare their distributions. Which plot do you think is appropriate? Plot the appropriate plot for each replicate, please plot them separately.
- Do not confuse the distribution of sample mean with the distribution of the sample (which you plotted in Part A).

For each set of replicates, I computed the sample mean across the 100 observations within each replicate. This resulted in 6 sets of sample means: 3 for the Uniform distribution (with 10, 100, and 1000 replicates) and 3 for the Pareto distribution (with 10, 100, and 1000 replicates). I'm guessing that having 9 of them is outdated, because 3 each from the 2 distributions is 6 sample means. The implementation is shown below:

```
def compute_sample_means(replicates_csv: str) -> np.ndarray:
    """
    Computes the sample mean for each replicate.

    :param replicates_csv: Path to CSV containing replicates
    :type replicates_csv: str
    :returns: Array of sample means, one per replicate
    :rtype: np.ndarray
    """
    df = pd.read_csv(replicates_csv)
    sample_means = df.mean(axis=0).to_numpy()
    return sample_means
```

Figure 5: Function for computing sample means from replicates.

To visualize the distribution of sample means, I chose to use histograms. This choice is best because histograms effectively visualize the shape and spread of the distribution mass of sample means, allowing us to best assess whether they appear to be approaching a normal distribution as the Central Limit Theorem predicts for larger numbers of replicate sample means. Since the whole point of this task is to gain insight about CLT, this plot ought to be the most appropriate to appreciate the convergence to Normal. The plotting function is shown below:

```
def plot_sample_means_distribution(sample_means: np.ndarray, dist_name: str,
                                  replicate_size: int, output_filename: str):
    """
    Plots the distribution of sample means.

    :param sample_means: Array of sample means
    :type sample_means: np.ndarray
    :param dist_name: Name of the distribution
    :type dist_name: str
    :param replicate_size: Replicate size
    :type replicate_size: int
    :param output_filename: Output filename for figure
    :type output_filename: str
    """
    sns.set_style("whitegrid")
    sns.set_palette("muted")
    plt.figure(figsize=(10, 6))

    ax = sns.histplot(sample_means, bins=50, stat='density',
                      edgecolor='white', alpha=0.7)

    ax.set_xlabel('Sample Mean', fontsize=14, labelpad=15)
```

```

ax.set_ylabel('Density', fontsize=14, labelpad=15)
ax.set_title(f'Distribution of {dist_name} Sample Means of Replicate with size {replicate_size}',
            fontsize=16, pad=20)

mean_of_means = np.mean(sample_means)
ax.axvline(mean_of_means, color='red', linestyle='--', linewidth=2,
            label=f'Mean of means: {mean_of_means:.3f}')
ax.legend(fontsize=11)

plt.tight_layout()
os.makedirs("../figures", exist_ok=True)
plt.savefig(f"../figures/{output_filename}", bbox_inches='tight', dpi=300)

```

Figure 6: Visualization function for plotting distribution of sample means.

These are the separate corresponding plots for each distribution and size:

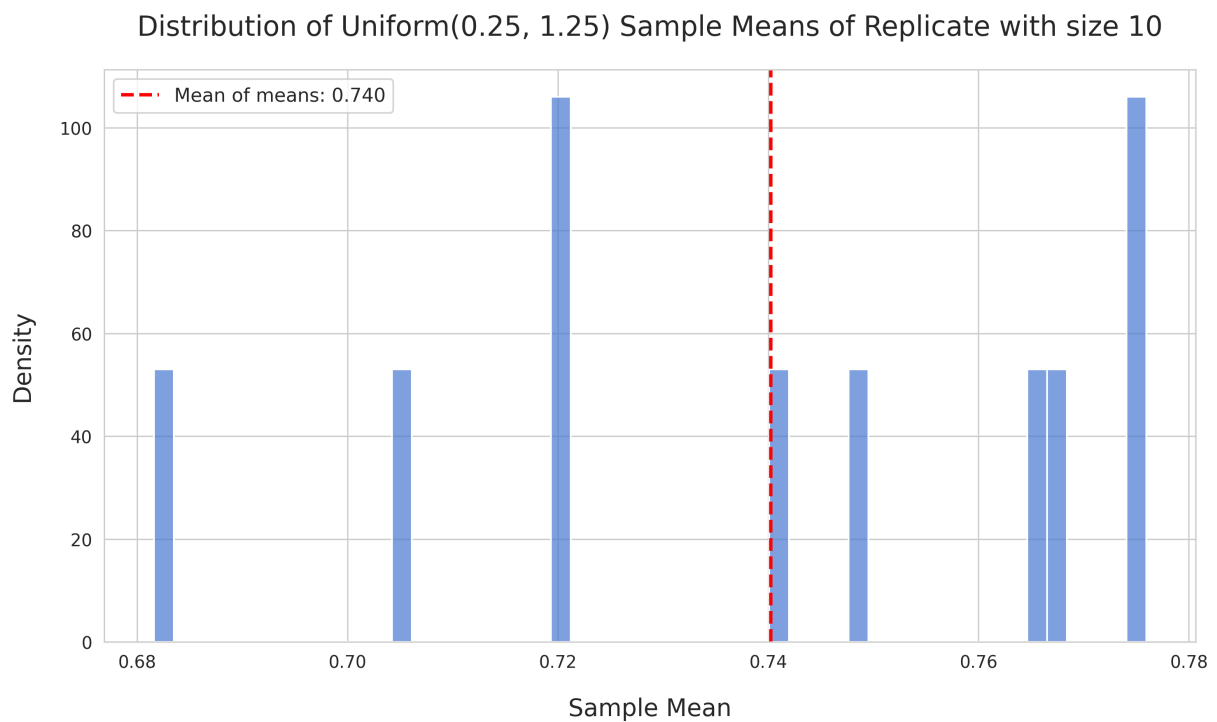


Figure 7: Distribution of sample means from Uniform(0.25, 1.25) with replicate size 10.

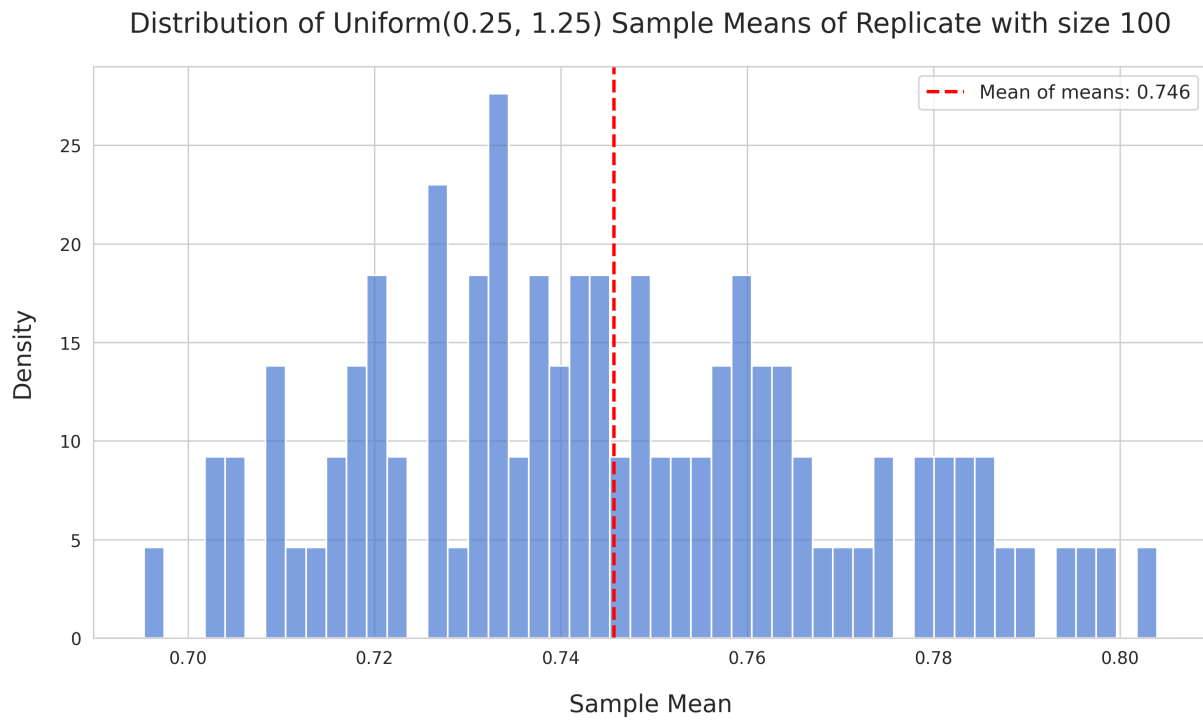


Figure 8: Distribution of sample means from Uniform(0.25, 1.25) with replicate size 100.

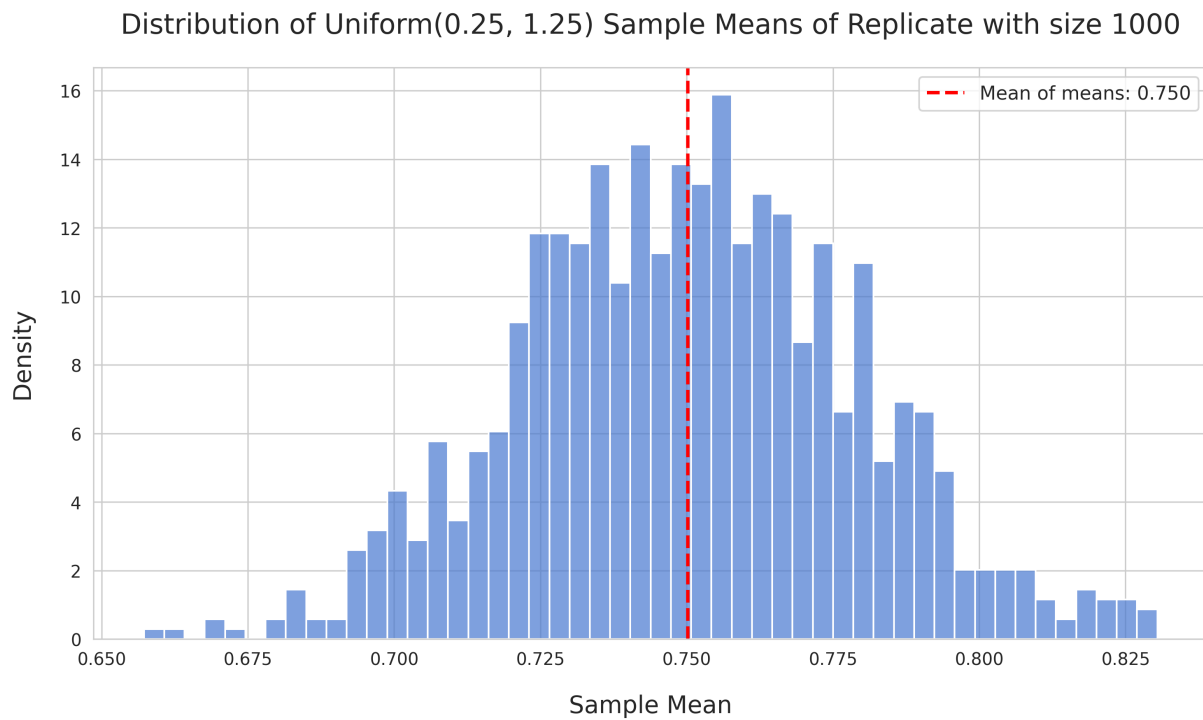


Figure 9: Distribution of sample means from Uniform(0.25, 1.25) with replicate size 1000.

Regarding the Uniform sample mean plots, we see the first plot begin with a very rough estimate of population mean from just 10 separate datasets, and we do not yet see a clear distribution. The second plot starts to take on form and shows a more accurate estimation of population mean. We can see the general shape of a Gaussian, which is then confirmed in the third plot, centered around 0.75, equal to $\frac{1.25+0.25}{2}$.

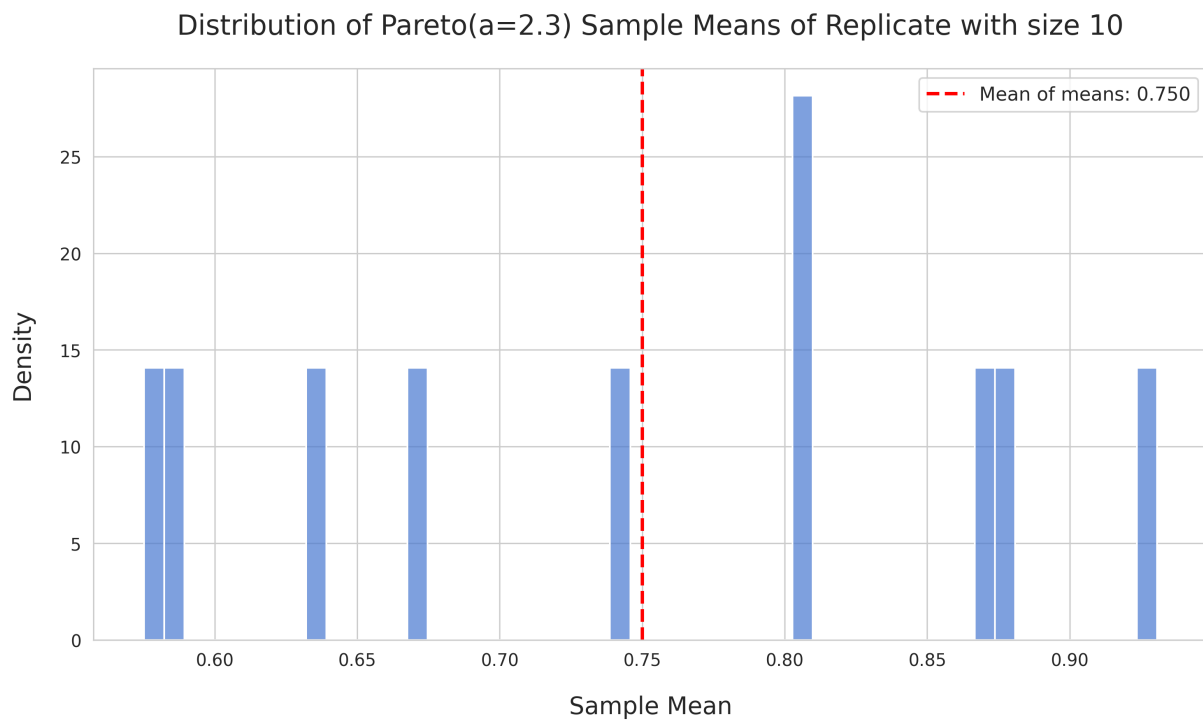


Figure 10: Distribution of sample means from Pareto($a=2.3$) with replicate size 10.

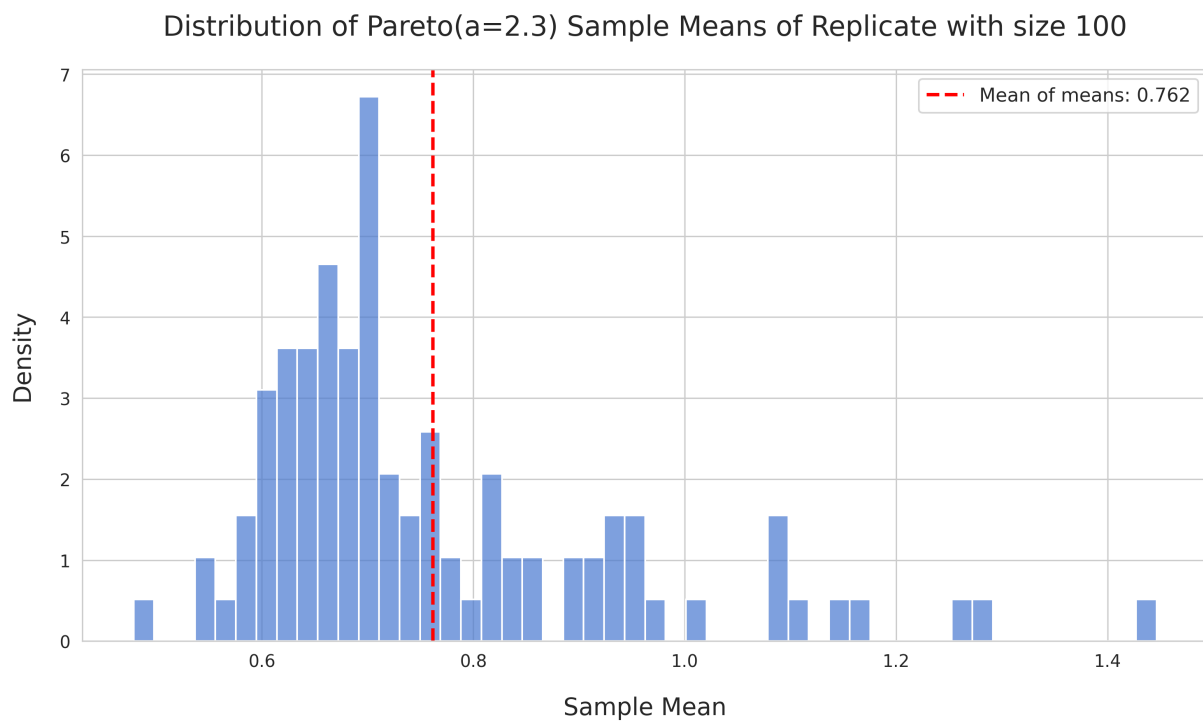


Figure 11: Distribution of sample means from Pareto($a=2.3$) with replicate size 100.

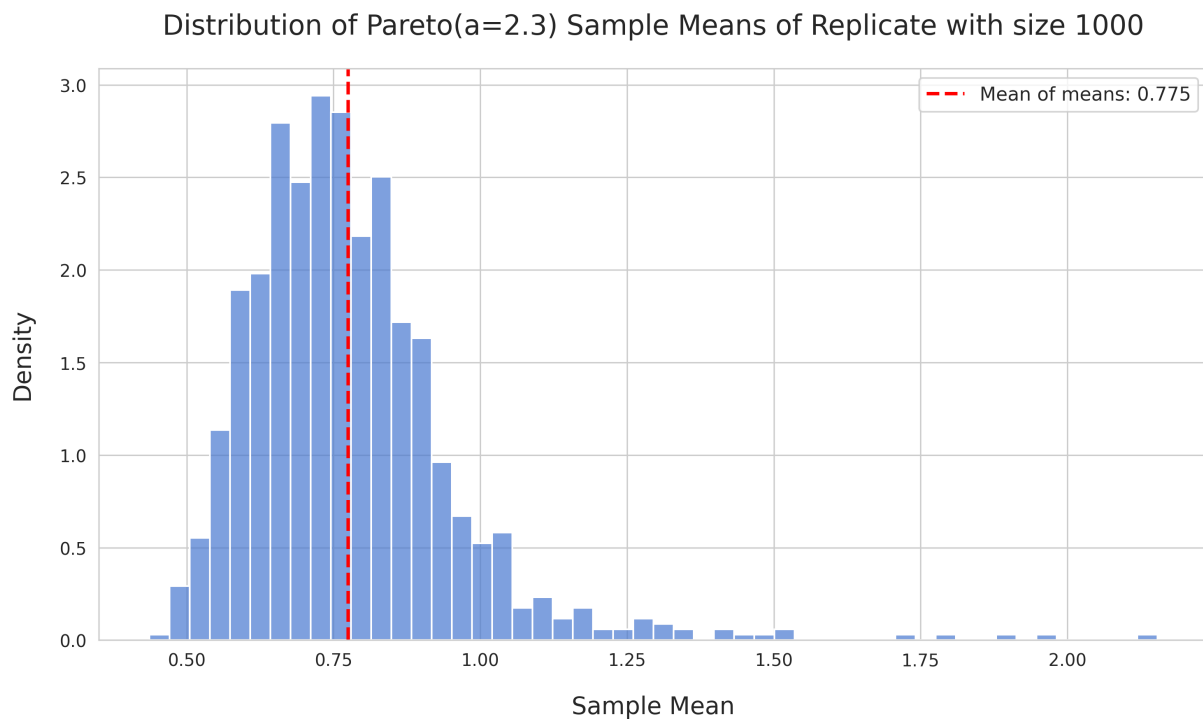


Figure 12: Distribution of sample means from Pareto($\alpha=2.3$) with replicate size 1000.

The Pareto sample mean plots show a similar convergence, though for the smaller two replicate sizes we have a number of datasets that tend to possess more of the predictable extreme values of the Power-Law distribution than others, causing a right skew in the distribution mass that mellows out only with increased number of sample means. The final plot shows a convergence toward a fairly smooth Gaussian, centered around 0.775, even if much slower due to its proclivity toward occasional extremes.

So regardless of the distribution type, we have at least a visual confirmation that CLT is applying appropriately to both, though the convergence of the sample means to a Gaussian as it relates to the number of samples may be influenced by the properties of the specific distributions in question.

3 Part C: Normal Approximation via CLT

- By the Central Limit Theorem, the distributions of sample means should approximate normal distributions. Determine the parameters (mean and standard deviation) of the approximating normal distributions for both power law and uniform cases.
- Overlay the probability density functions (PDFs) of these normal distributions on the corresponding plot from part B.

I computed the sample mean and the standard deviation of the sample means for each set of replicates, and then overlaid the PDF of the corresponding Gaussian defined with these (mean, std.) pairs on top of each of their respective plots from part B.

According to the Central Limit Theorem, for samples of size n drawn from a distribution with mean μ and variance σ^2 , the distribution of sample means should approximate a normal distribution with mean μ and standard error $SE = \frac{\sigma}{\sqrt{n}}$.

For the Uniform(0.25, 1.25) distribution, the theoretical parameters are:

$$\mu = \frac{0.25 + 1.25}{2} = 0.75$$

$$\sigma = \frac{1.25 - 0.25}{\sqrt{12}} \approx 0.289$$

$$SE = \frac{0.289}{\sqrt{100}} \approx 0.029$$

And the Pareto($a=2.3$) distribution:

$$\mu = \frac{1}{a-1} = \frac{1}{1.3} \approx 0.769$$

$$\sigma^2 = \frac{1}{(a-1)^2(a-2)} = \frac{1}{(1.3)^2(0.3)} \approx 1.97 \Rightarrow \sigma \approx 1.40$$

$$SE = \frac{1.40}{\sqrt{100}} \approx 0.140$$

The observed parameters from the sample means for each replicate size are shown below:

Observed Parameters from Sample Means (Mean μ , Standard Deviation σ)

Replicate Size	Uniform Distribution		Power Law Distribution	
	Mean (μ)	Std Dev (σ)	Mean (μ)	Std Dev (σ)
10	0.740	0.031	0.750	0.121
100	0.746	0.025	0.762	0.174
1000	0.750	0.029	0.775	0.176
Theoretical	0.750	0.029	0.769	0.140

For the Uniform distribution, the observed standard deviation stabilizes around 0.029, matching the theoretical standard error. For the Pareto distribution, the observed standard deviation increases from 0.121 to 0.176 as we get more replicates. The initial underestimation with 10 replicates likely occurs because we don't have enough samples to capture the full variability from the heavy tail, whereas the 1000 replicates provide a more realistic understanding of the variability caused by the occasional extremes. The observed means also converge: the Uniform mean reaches exactly 0.750 with 1000 replicates, and the Pareto mean reaches 0.775, fairly close to the theoretical 0.769.

This demonstrates that while the theoretical distribution of sample means is fixed (determined by the original distribution and sample size $n = 100$), we need a sufficiently high number of replicates to accurately estimate its parameters, especially for heavy-tailed distributions like the Pareto.

The function for computing and overlaying the normal PDFs is shown below:

```

def overlay_normal(sample_means: np.ndarray, dist_name: str,
                   replicate_size: int, output_filename: str):
    """
    Plots the histogram of sample means with overlaid normal PDF.

    :param sample_means: Array of sample means
    :type sample_means: np.ndarray
    :param dist_name: Name of the distribution
    :type dist_name: str
    :param replicate_size: Replicate size
    :type replicate_size: int
    :param output_filename: Output filename for figure
    :type output_filename: str
    """
    sns.set_style("whitegrid")
    sns.set_palette("muted")
    plt.figure(figsize=(10, 6))

    ax = sns.histplot(sample_means, bins=50, stat='density',
                      edgecolor='white', alpha=0.7, label='Sample Mean Distribution')

    mu = np.mean(sample_means)
    sigma = np.std(sample_means)
    x = np.linspace(min(sample_means), max(sample_means), 500)
    pdf = stats.norm.pdf(x, loc=mu, scale=sigma)

    plt.plot(x, pdf, 'r--', label=f'Normal Approx. PDF\(\mu={mu:.3f}, \sigma={sigma:.3f}\)')

    ax.set_xlabel('Sample Mean', fontsize=14, labelpad=15)
    ax.set_ylabel('Density', fontsize=14, labelpad=15)
    ax.set_title(f'Normal Overlay for {dist_name} Sample Means of Replicate Size {replicate_size}',
                 fontsize=16, pad=20)

    plt.legend(fontsize=11)
    plt.tight_layout()
    os.makedirs("../figures", exist_ok=True)
    plt.savefig(f"../figures/{output_filename}", bbox_inches='tight', dpi=300)

```

Figure 13: Function for overlaying fitted normal PDF onto the sample mean distributions.

Normal Overlay for Uniform(0.25, 1.25) Sample Means of Replicate Size 10)

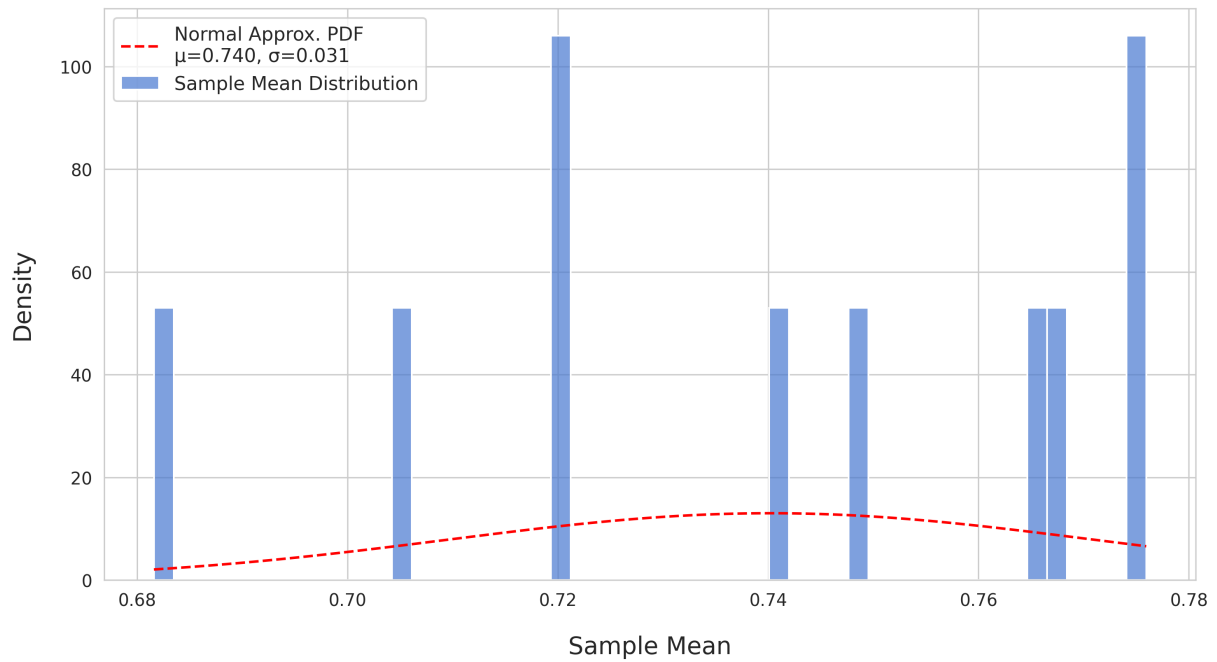


Figure 14: Uniform sample means, replicate size 10 with normal PDF overlay.

Normal Overlay for Uniform(0.25, 1.25) Sample Means of Replicate Size 100)

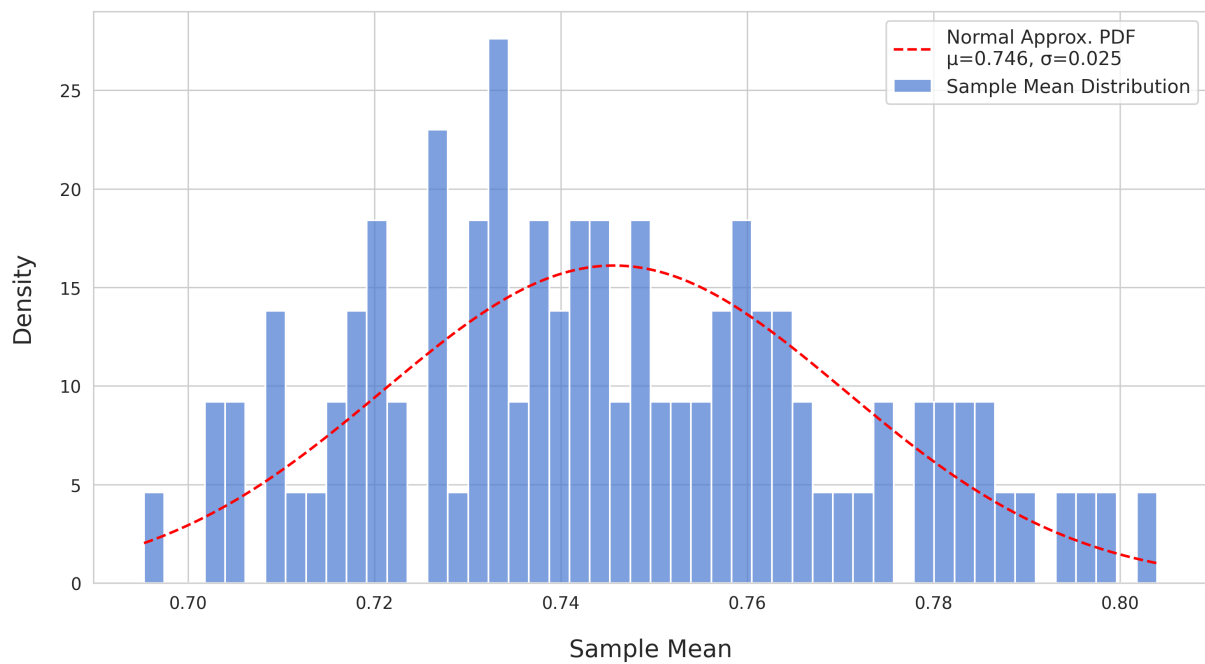


Figure 15: Uniform sample means, replicate size 100 with normal PDF overlay.

Normal Overlay for Uniform(0.25, 1.25) Sample Means of Replicate Size 1000)

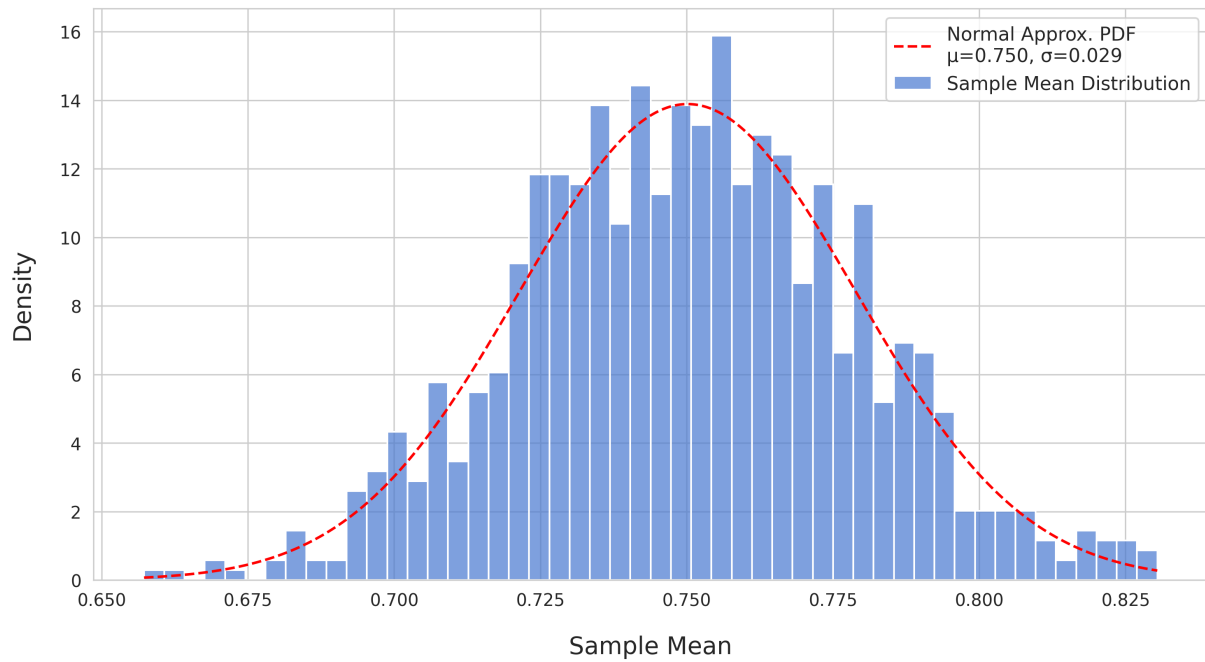


Figure 16: Uniform sample means, replicate size 1000 with normal PDF overlay.

Normal Overlay for Pareto($a=2.3$) Sample Means of Replicate Size 10)

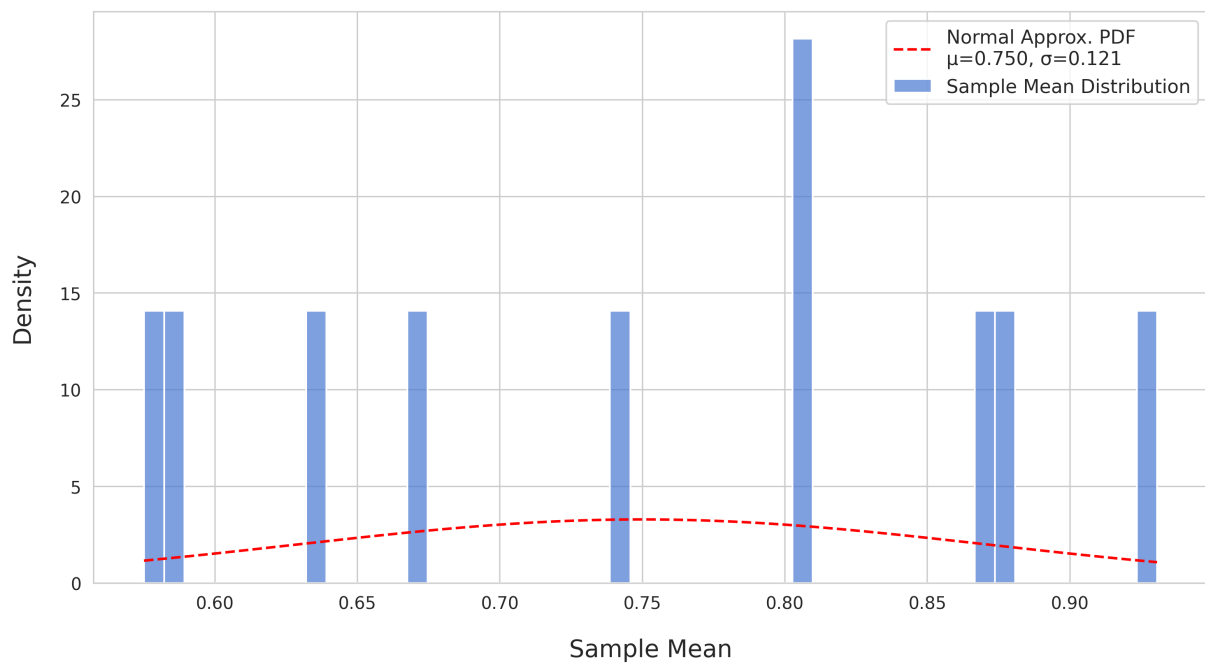


Figure 17: Power-law sample means, replicate size 10 with normal PDF overlay.

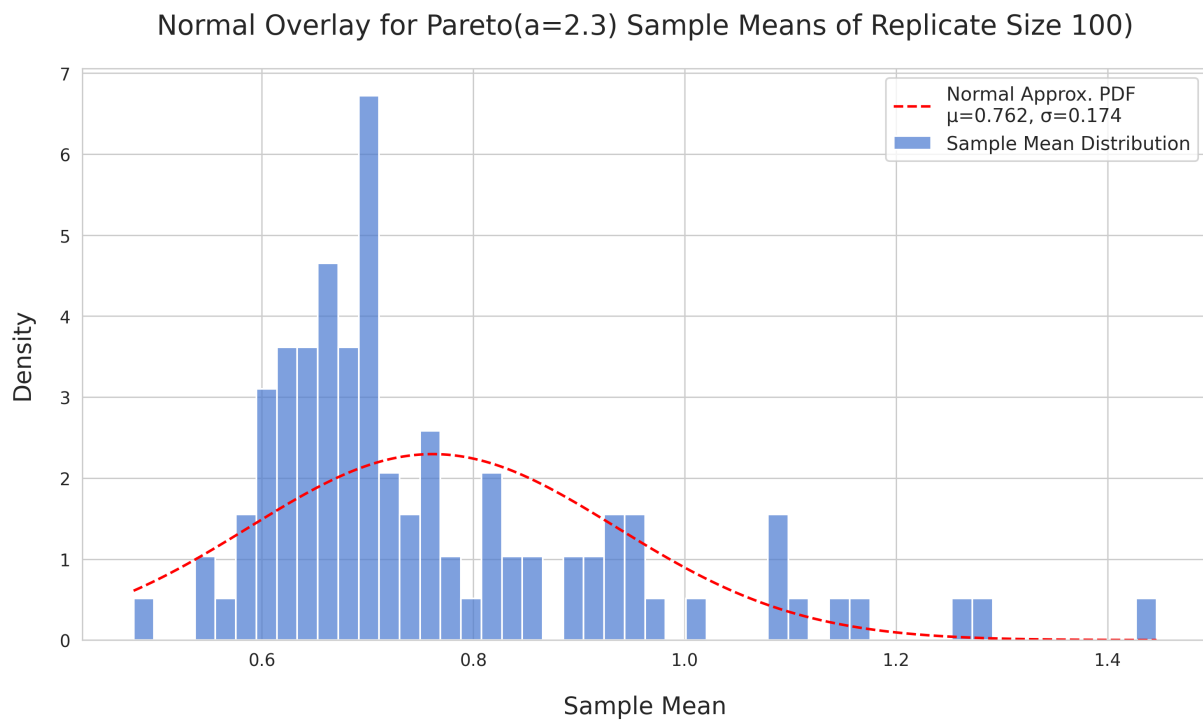


Figure 18: Power-law sample means, replicate size 100 with normal PDF overlay.

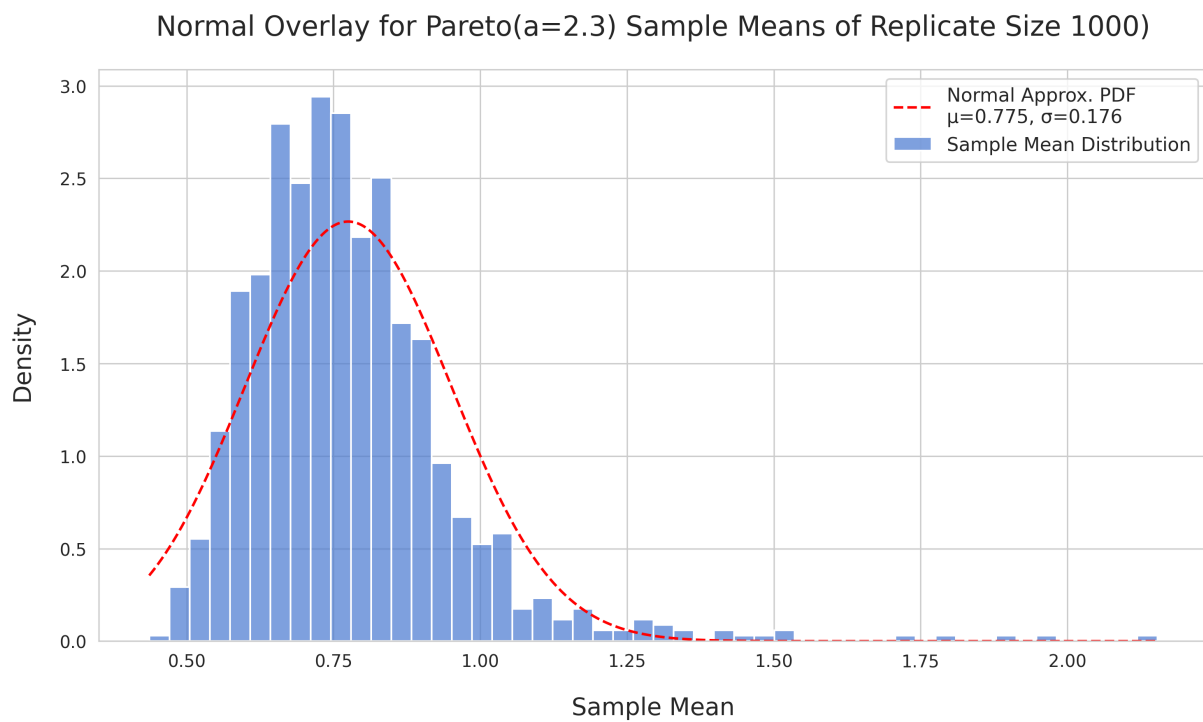


Figure 19: Power-law sample means, replicate size 1000 with normal PDF overlay.

4 Part D: Distribution Comparison with KS-Test

- Compare the empirical distributions of sample means (from power law vs. uniform) using the Kolmogorov–Smirnov (KS) test.
- Discuss whether the KS-test supports the theoretical expectation that both sample mean distributions should approximate normality under the CLT. And investigate how the power of the KS-test changes as the number of sample means increases (10, 100, 1000).

I performed one-sample KS tests comparing each distribution of sample means to its theoretical normal distribution. The implementation is shown below:

```
def ks_test_theoretical_normal(sample_means: np.ndarray, theoretical_mu: float,
                               theoretical_se: float) -> tuple:
    """
    Performs KS test comparing sample means to their theoretical normal distribution.

    :param sample_means: Array of sample means
    :type sample_means: np.ndarray
    :param theoretical_mu: Theoretical mean from CLT
    :type theoretical_mu: float
    :param theoretical_se: Theoretical standard error from CLT
    :type theoretical_se: float
    :returns: Tuple of KS statistic, p-value
    :rtype: tuple
    """
    ks_stat, p_value = stats.kstest(sample_means,
                                     lambda x: stats.norm.cdf(x,
                                                                loc=theoretical_mu,
                                                                scale=theoretical_se))

    return ks_stat, p_value
```

Figure 20: KS test function comparing sample means to theoretical normal distribution.

The one-sample KS test results for each distribution at different replicate sizes are tabulated below. Each test compares the empirical distribution of sample means to the theoretical normal distribution (Uniform: $N(0.75, 0.029^2)$, Pareto: $N(0.769, 0.140^2)$):

KS Test Results: Sample Means vs Theoretical Normal Distribution

Replicate Size	Uniform Distribution		Power Law Distribution	
	KS Statistic	p-value	KS Statistic	p-value
10	0.240	0.537	0.160	0.925
100	0.114	0.135	0.221	8.92e-5
1000	0.016	0.945	0.059	0.002

The Uniform distribution shows support for normality across each replicate size, with all the p-values being above 0.05, failing to reject the null hypothesis; the sample means ought to be following a Normal distribution. The KS statistics decreases from 0.240 to 0.016 with increasing replicates, so the agreement with the theoretical Normal distribution is increasing with greater numbers of sample means, as CLT would suggest.

The Pareto distribution reveals the effect of heavy tails on the KS test's power. At replicate size 10, the test fails to reject ($p = 0.925$) with a KS statistic of 0.160. This signals the limits of the KS test's power at smaller sample sizes because the KS stat at replicate size 10 is 0.16, which is larger than at replicate size 1000, despite the p-value at replicate size 1000 being significant while the other is not. For this reason, the KS test alone is not as powerful until sample size and statistical power at that sample size are taken into account, because in the case of size 10, the null could be false, but the test does not have enough power at this scale to detect it, whereas at size 1000 the test has so much power that it can detect any of the remaining deviations.

We see at size 100, the KS statistic actually increases to 0.221 and becomes highly significant ($p = 8.92 \times 10^{-5}$), rejecting the null hypothesis. This increase shows that with 100 replicates, the KS test has enough power at this scale to detect the sample's disagreement with the Gaussian from the heavy tail's influence on the sample mean distribution. This marks a turning point of the KS test becoming powerful with respect to our observations, and then the increase to size 1000 confirms the convergence, albeit the deviations from Normal behavior are still apparent and detectable by the test.

I also went ahead and plotted the convergence of the KS statistics for each distribution across the replicate sizes. Although the plot looks quite simple as a visual, it illustrates pretty well the behavior I just mentioned:

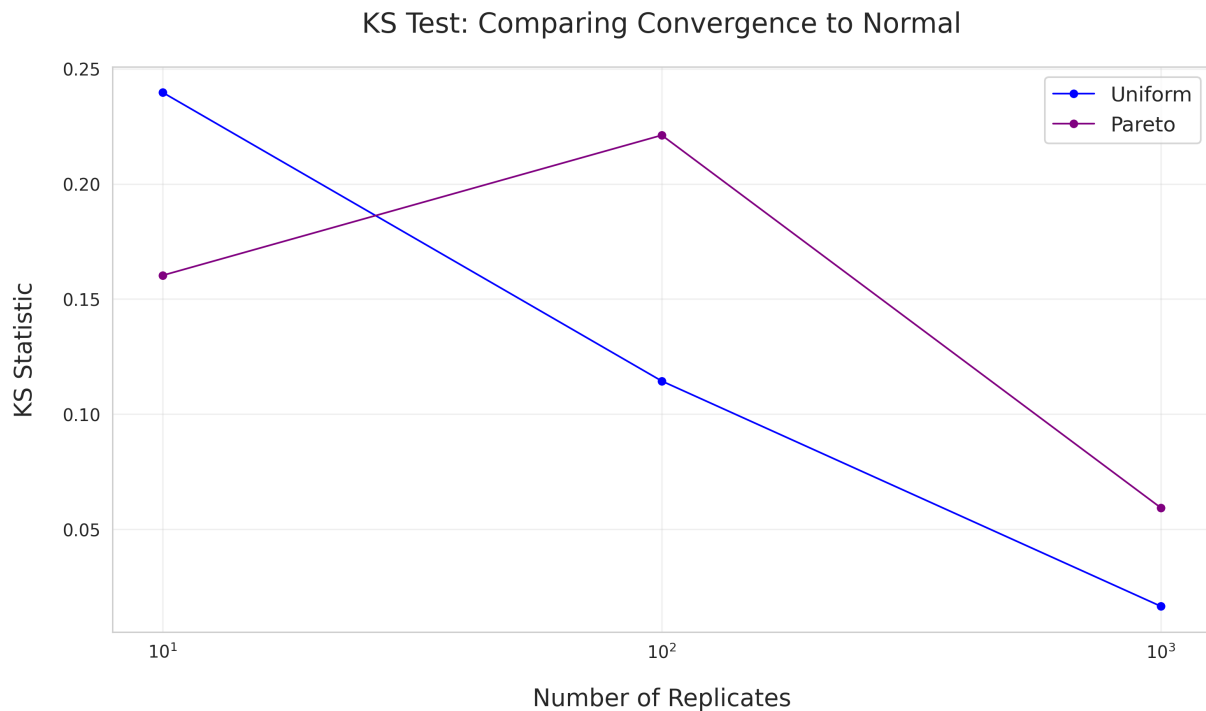


Figure 21: KS statistics vs replicate size, comparing Uniform and Pareto convergence.

Regarding the Uniform, it shows a consistently decreasing KS statistic, reaching almost zero by size 1000. It has excellent agreement with the Normal simply due to its construction: it doesn't have the issues in collecting sample means with similar behavior like the Pareto does. The Pareto shows an increase from 0.16 to 0.221 between size 10 to 100, and then decreases to 0.059 at size 1000. This is showing that initially, more data to reference the cumulative structure off of shows that deviations were present but undetectable. Then the CLT effect dominates over the tail behaviors of the Pareto causing the occasional extreme sample means.

The KS test results support the Central Limit Theorem's prediction that both distributions converge toward the theoretical Normal, though with the caveat that the rate of convergence will be influenced by the particular tail behaviors of the distributions in question.

Task 3: Hypothesis Testing

COURSE NAME

CSDS 413 Introduction to Data Analysis

Authors:

Jacob Anderson

October 13, 2025

Contents

Context	2
1 Part A	3
2 Part B	4
3 Part C	5
4 Part D	9
5 Part E	10

Context

Car sales people usually claim that Japanese cars have better mileage than US cars. As a critical thinker, you know not to immediately believe what somebody who is trying to sell you something says. However, as a data scientist, you are curious to see whether this claim has any statistical basis.

You found some data collected in “cars.csv”. The first column contains a sample of measured mileage per hour for different US cars and the second column contains a sample of measured mileage per hour for different Japanese Cars. Answer the following questions:

- (a) How many US and how many Japanese cars does the dataset contain?
- (b) State the hypothesis you are testing to scrutinize the sales person's claim.
- (c) What is the statistic you would compute to test this hypothesis and its theoretical distribution?
- (d) What is the value of your statistic and what is its p-value?
- (e) What is your conclusion about the sales person's claim?

Note: Be sure to check any conditions or assumptions of the chosen test before computing statistics and drawing your conclusion.

1 Part A

How many US and how many Japanese cars does the dataset contain?

From the context, "first column contains a sample of measured mileage per hour for different US cars and the second column contains a sample of measured mileage per hour for different Japanese Cars". This ought to mean that each row, regardless of if it contains a mileage that has been seen before, represents a different car for the purpose of counting. Below is a little Python REPL to count the number of rows for each column containing a mileage:

```
(assignment2-py3.12) jande@BT-7274:~/csds413_assignments/assignment2/
task3_hypothesis_testing/datasets$ python
Python 3.12.0 (main, Jun  9 2025, 09:52:12) [GCC 13.3.0] on linux
Type "help", "copyright", "credits" or "license" for more information.
>>> import pandas as pd
>>> cars = pd.read_csv("cars.csv", header=None)
>>> us_cars = len(cars[cars[0].notna()])
>>> us_cars
249
>>> jp_cars = len(cars[cars[1].notna()])
>>> jp_cars
79
```

Figure 1: Python REPL counting the number of cars of each type in the dataset.

There are 249 US cars and 79 Japanese cars.

2 Part B

State the hypothesis you are testing to scrutinize the sales person's claim.

H_0 : The average mileage of Japanese cars is less than or equal to the average mileage of US cars.

H_1 : The average mileage of Japanese cars is greater than the average mileage of US cars.

3 Part C

What is the statistic you would compute to test this hypothesis and its theoretical distribution?

I'm going to reasonably assume the measurement of US car mileages have no dependency on Japanese car mileages and vice versa. That being said, the hypothesis in part B is a statement on the comparison of average mileage, which I framed in that manner because the claim from the problem context comments on what people usually claim about these cars, so a comparison of average mileage of these cars ought to be most appropriate.

With that established, choosing an appropriate test to compare the means of two independent groups immediately brings two-sample Student's t-test to mind. This test requires that the groups be independent, that the samples should be about Normal, and their variances should be about the same. Below is the function and corresponding Q-Q Plot I generated to see how closely each group resembles a Gaussian:

```
def main():
    data = pd.read_csv('../datasets/cars.csv', header=None)
    us_mileage = data[0].dropna().to_numpy()
    jp_mileage = data[1].dropna().to_numpy()

    sns.set_style("whitegrid")
    sns.set_palette("muted")
    _, axes = plt.subplots(1, 2, figsize=(12, 6))

    percentiles = np.linspace(0, 100, 75)
    norm_quantiles_us = np.percentile(np.random.normal(loc=np.mean(us_mileage),
    scale=np.std(us_mileage), size=10_000), percentiles)
    sample_quantiles_us = np.percentile(us_mileage, percentiles)

    norm_quantiles_jp = np.percentile(np.random.normal(loc=np.mean(jp_mileage),
    scale=np.std(jp_mileage), size=10_000), percentiles)
    sample_quantiles_jp = np.percentile(jp_mileage, percentiles)

    axes[0].scatter(norm_quantiles_us, sample_quantiles_us, alpha=0.5,
                     s=20, color='blue', edgecolor='blue')
    axes[0].plot([min(norm_quantiles_us), max(norm_quantiles_us)],
                 [min(norm_quantiles_us), max(norm_quantiles_us)],
                 'k--', lw=2, alpha=0.7)
    axes[0].set_title("Q-Q Plot: US Cars Mileage", fontsize=14)
    axes[0].set_xlabel("Theoretical Quantiles", fontsize=12)
    axes[0].set_ylabel("Sample Quantiles", fontsize=12)

    axes[1].scatter(norm_quantiles_jp, sample_quantiles_jp, alpha=0.5,
                     s=20, color='purple', edgecolor='purple')
    axes[1].plot([min(norm_quantiles_jp), max(norm_quantiles_jp)],
                 [min(norm_quantiles_jp), max(norm_quantiles_jp)],
                 'k--', lw=2, alpha=0.7)
    axes[1].set_title("Q-Q Plot: Japanese Cars Mileage", fontsize=14)
    axes[1].set_xlabel("Theoretical Quantiles", fontsize=12)
    axes[1].set_ylabel("Sample Quantiles", fontsize=12)

    plt.tight_layout()
    plt.savefig(f"../figures/qqplot.png", bbox_inches='tight', dpi=300)
```

Figure 2: Function for plotting theoretical against sample quantiles for US and Japanese cars.

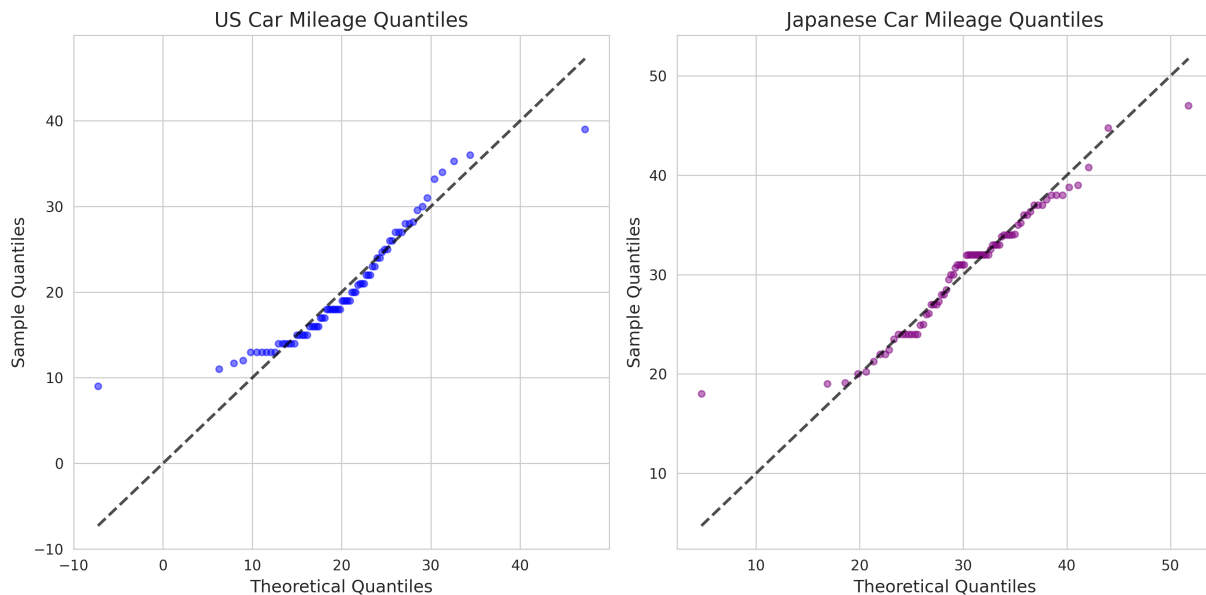


Figure 3: Q-Q plots for US and Japanese cars quantiles.

The quantiles for both the US cars and the Japanese cars stay close to the agreement line through the Q-Q Plot, especially toward the middle quantiles, though they do both have a couple that visually deviate quite far from the agreement line in the more extreme cases. The plot seems to suggest they are both approximately Normal, but it would be helpful to also visualize the distribution shape directly as well, not to mention they could be plotted on the same figure to give a general idea of which country may be produce higher mileage cars in the average-case:

```
def main():
    data = pd.read_csv('../datasets/cars.csv', header=None)
    us_mileage = data[0].dropna()
    jp_mileage = data[1].dropna()

    sns.set_style("whitegrid")
    sns.set_palette("muted")

    plt.figure(figsize=(10, 6))

    sns.histplot(us_mileage, bins=30, kde=False, color='blue', edgecolor='white',
alpha=0.6, label='US Cars')
    sns.histplot(jp_mileage, bins=30, kde=False, color='red', edgecolor='white',
alpha=0.6, label='Japanese Cars')

    plt.title('Histogram: US vs Japanese Cars Mileage', fontsize=16)
    plt.xlabel('Mileage', fontsize=14)
    plt.ylabel('Count', fontsize=14)
    plt.legend(fontsize=12)

    plt.tight_layout()
    os.makedirs("../figures", exist_ok=True)
    plt.savefig("../figures/histogram.png", bbox_inches='tight', dpi=300)
```

Figure 4: Function for plotting US and Japanese car mileage distributions.

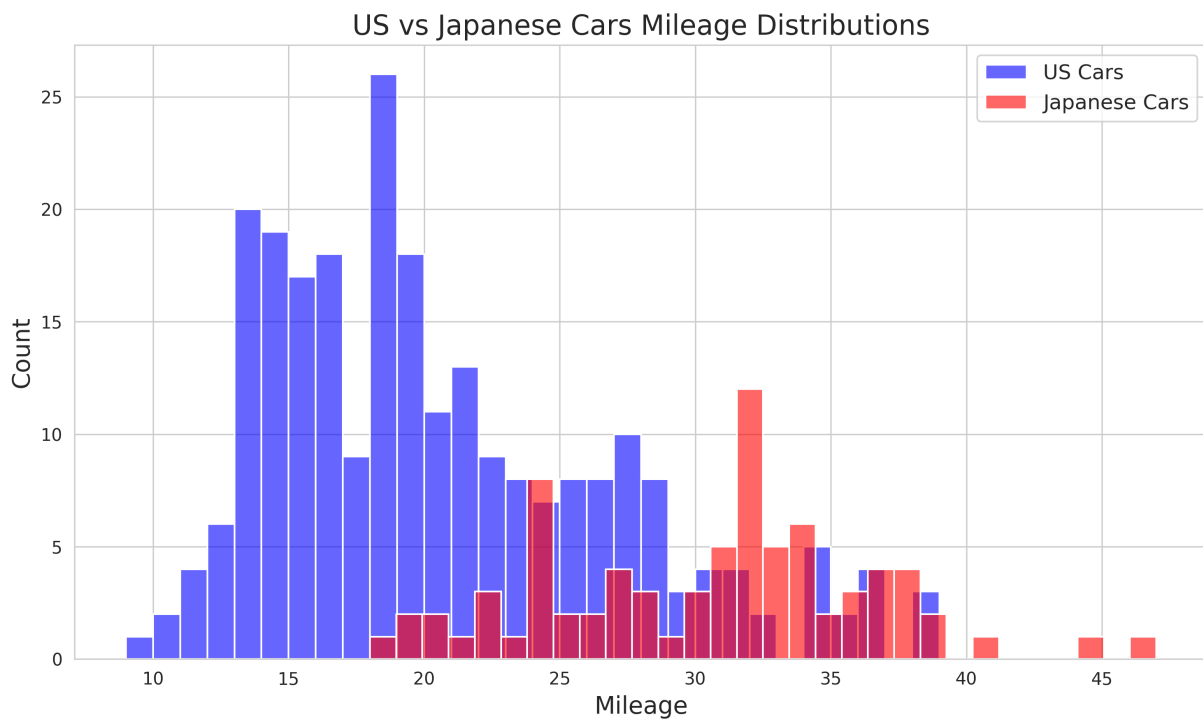


Figure 5: Histogram plots for US and Japanese cars mileages.

This result gives us pretty decent visual confirmation that these groups both approximate Normal distributions, and the addition histogram plot does well to visualize the peculiarities in the distribution mass that attribute to the deviations from the agreement line that we saw in the Q-Q Plot. For example, the heavier left tail of the US car distribution corresponding to the deviations above the agreement line or the right tail tapering off a bit faster than the right in the case of Japanese car mileages corresponding to the deviations below the agreement line. Also like I mentioned, the histogram gives us a rough confirmation that the average Japanese car mileage is higher than the US cars.

We need to also confirm that the variance are about the same, so for this purpose I produced box plots corresponding to each country:

```
def main():
    data = pd.read_csv('../datasets/cars.csv', header=None)
    us_mileage = data[0].dropna()
    jp_mileage = data[1].dropna()

    df = pd.DataFrame({
        'Mileage': pd.concat([us_mileage, jp_mileage], ignore_index=True),
        'Region': ['US'] * len(us_mileage) + ['Japan'] * len(jp_mileage)
    })

    sns.set_style("whitegrid")
    sns.set_palette("muted")

    plt.figure(figsize=(8, 6))
    ax = sns.boxplot(x='Region', y='Mileage', data=df, showfliers=True)

    ax.set_title('Car Mileages for US vs Japanese Cars', fontsize=16)
    ax.set_xlabel('')
    ax.set_ylabel('Mileage', fontsize=12)

    plt.tight_layout()
    plt.savefig("../figures/boxplot.png", bbox_inches='tight', dpi=300)
```

Figure 6: Function for plotting US and Japanese car mileages as box plots.

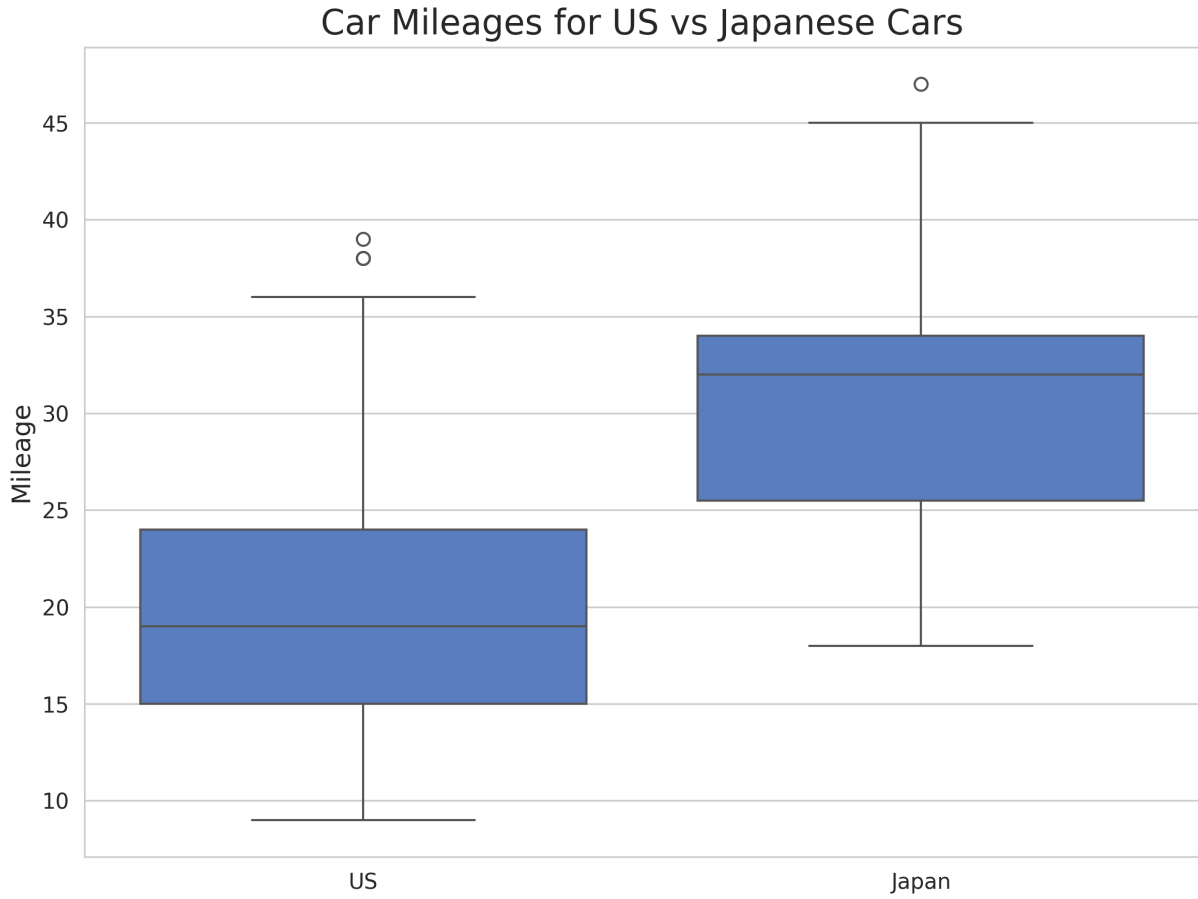


Figure 7: Box plots for US and Japanese cars mileages.

The box plots show a very promising signal that their inter-quartile ranges are about the same size and it shows that their whiskers protrude by about the same amount both above and below their IQR. We can also see a comparison of the medians from this visual, and given that the respective distributions are sufficiently symmetrical, then we have another rough signal that the Japanese cars may greater mileage on average.

That being said, these visualizations confirm that the underlying conditions of the t-test have been adequately met. This test statistic follows a t-distribution, which models the sampling distribution of the standardized difference between sample means. It is similar to the Normal distribution but with heavier tails, especially for smaller sample sizes, reflecting an increased uncertainty. In this case, the t-distribution can be expressed as:

$$t = \frac{\bar{x}_{jp} - \bar{x}_{us}}{\sigma_{\bar{x}_{jp} - \bar{x}_{us}}}$$

where $\sigma_{\bar{x}_{jp} - \bar{x}_{us}}$ is the standard error of the difference between sample means:

$$\sigma_{\bar{x}_{jp} - \bar{x}_{us}} = \sqrt{\frac{\sigma_{jp}^2}{n_{jp}} + \frac{\sigma_{us}^2}{n_{us}}}$$

This test statistic follows this distribution with $n_{jp} + n_{us} - 2$ degrees of freedom under H_0 . Because H_1 is one-sided (Japanese > US), I will perform a right-tailed test, rejecting H_0 if the t-statistic is sufficiently large.

4 Part D

What is the value of your statistic and what is its p-value?

```
def main():
    data = pd.read_csv('../datasets/cars.csv', header=None)

    us_mileage = data[0].dropna().to_numpy()
    jp_mileage = data[1].dropna().to_numpy()

    n_us = len(us_mileage)
    n_jp = len(jp_mileage)

    mean_us = np.mean(us_mileage)
    mean_jp = np.mean(jp_mileage)

    var_us = np.var(us_mileage, ddof=1)
    var_jp = np.var(jp_mileage, ddof=1)
    se = np.sqrt(var_us/n_us + var_jp/n_jp)

    t_stat = (mean_jp - mean_us) / se
    df = n_us + n_jp - 2
    p_value = stats.t.sf(t_stat, df)

    print(f"t-statistic: {t_stat:.4f}")
    print(f"p-value: {p_value:.4e}")
```

Figure 8: Function for computing the t statistic and its p-value.

The t-statistic was reported by the function as 12.946, with a p-value of 1.6197×10^{-31} .

5 Part E

What is your conclusion about the sales person's claim?

The t-statistic of 12.946 indicates that the observed difference in mean is nearly 13 standard errors away from zero. Pairing this result with a p-value of 1.6197×10^{-31} suggests that if the null hypothesis were true, the probability of observing a t-statistic this high would be virtually zero. This provides overwhelming evidence in favor of rejecting the null hypothesis and concluding that the salesperson is correct in claiming Japanese car mileage is better than US car mileage.